

从经典例题到 B 题：七天答辩入门教材

先学方法，再做赛题，最后读源码

目录

从经典例题到 B 题代码：七天入门课	2
第 1-2 课 从坐标到凸包与角楔	5
第 3 课 直径与最小包围圆	24
第 4 课 从“平均不错”走到“最坏也尽量好”	33
第 5 课 覆盖、旅行商与启发式：先分清要解决什么	42
第 6-7 课 有误差时保留哪些可能，没看到时又能排除什么	53
附录 把课本方法讲到自己的程序上	66

从经典例题到 B 题代码：七天入门课

这本教材给第一次接触计算几何、鲁棒优化和在线搜索的队员使用。目标不是七天内掌握这些学科的全部研究成果，而是在答辩中能独立说明：这个方法处理什么输入，为什么这样算，结论有什么前提，程序在哪里实现，遇到反例怎么办。

读法：先学一门方法，再做本题练习

每课都按同一顺序组织：

1. 认识问题。暂时不看干扰源和代码，先用一个平常问题说明为什么需要该方法。
2. 建立直观。画图或用可手算的小数字，看对象怎样变化。
3. 标准方法。交代通行名称、定义、步骤、正确性理由与适用条件。教材例题给出可查的原出处；文中的解法和讲述重新编写，不把自编数据冒充原书例题。
4. 自己动手。先做一个同类型小题，再看逐步解答。能纸笔重做才进入下一段。
5. 迁移到 B 题。本题的题目放在课后应用：明确哪些条件与标准例题相同，哪些是额外的误差、安全、有限预算、在线反馈或定向盲区。
6. 回到程序。在源码中找到输入、算法主体、输出、下一层调用；区分严格证书与数值试探。

在课文里看到“经典例题”，指公开教材或作者文档里有明确出处的题。看到“教学小例”，指为了练步骤在此编写的小数值题。看到“B 题练习”，才开始处理自己的比赛题。三者不可互换。

七天，每天五小时

天	基础课与原例	B 题迁移练习	当晚闭卷验收
1	第 1 课：坐标、角度、叉积、凸性；第 2 课前半：凸包	一条示向线为什么变成允许区域	用图讲出两条边界及角度回绕，不看代码
2	第 2 课后半：半平面交；第 3 课：直径、旋转卡壳、最小包围圆	Q1 多次读数的交集、区域直径、直径圆反例	手算小四边形，解释直径和包围圆的不同
3	第 4 课：约束优化、极小极大、离散与连续	Q2 安全站域与最坏读数	写准先选站、后发生报告的量词顺序；指出数值近似在哪里
4	第 5 课：覆盖、路线问题和启发式；第 6 课前半：集合估计	Q3 为什么能找齐、怎样保留真值	手工更新一个候选集合，说明九点覆盖的前提

天	基础课与原例	B 题迁移练习	当晚闭卷验收
5	第 6 课后半：主动观测与清除证书；第 7 课：定向观测	Q3 安全清除、Q4 双负观测	举出“单次无信号不能排除定向源”的反例
6	综合练习与源码精读	从四个入口各追踪一次函数调用	不看书运行固定种子短例，任指一段代码说清输入与作用
7	错题复做和模拟答辩	5 分钟与 8 分钟两种讲法	能承认近似与局限，正确区别本地运行、正式记录和数学证明

每日五小时按 90 分钟读与画图、80 分钟纸笔例题、70 分钟做本题练习和核答案、45 分钟追踪源码并运行、15 分钟口头复述 分配。做不出来就回到上一节，不靠记住标准答案掩盖概念缺口。

掌握程度：哪些要会证，哪些只需辨认

层级	应达到的表现	对应内容
A 必会	不看书画图、列式、算一个小例子，指出程序中的同一步	方位角、叉积符号、集合交、凸包、半平面裁剪、最小包围圆、覆盖与负观测逻辑
B 会解释	说清算法步骤、何时有效、时间或数值代价，拿标准例题和本题比较	旋转卡壳、随机增量包围圆、极小极大、TSP/插入启发式、集合估计
C 会辨界	知道名称与反例，不把没有实现的方法说成己方代码	贝叶斯滤波、Kalman 滤波、精确 TSP、一般集合覆盖最优、全局连续优化证明

“立体地理解”每个方法至少用三种表示：一张几何图、一遍纸笔数字过程、一处源码执行轨迹。题目本身的坐标与方向建模在二维平面；这里的“立体”是学习方式，并不把平面模型虚构成三维算法。

每课收尾的自测五问

1. 这类问题为什么会出现？输入和输出各是什么？
2. 标准方法的名字、关键步骤和成立条件是什么？能手算最小例子吗？
3. 原例的哪一项假设被 B 题改掉了？改动后新增了什么约束或风险？
4. 程序的哪个函数实施了同一个数学动作？函数前后哪个量发生变化？
5. 这项结论是严格证明、保守上界、数值近似，还是一次实验观察？

答不出前两问，先不要背后面的源码行号。行号是寻找证据的地图，不是方法的定义。

与旧版材料的关系

本目录是按教学顺序重写的主课。../book/ 中旧版合订本可作技术索引和深入查阅材料，尤其是函数表；不要按它的章节顺序零基础通读。原始可运行程序放在 ../source/，固定种子小例在 ../examples/。课程结束再用旧版第 5、6 章做代码抽问和答辩追问。

第 1-2 课 从坐标到凸包与角楔

这两课学什么，学到哪里就够了

拿一张纸，画几个点。我们准备依次解决三个很具体的问题：

1. 方向：从一个点看另一个点，它在什么方向？
2. 包围：有一堆点，怎样用一根绷紧的橡皮筋围住它们？
3. 筛选：一个位置要同时满足几条条件，最后剩下哪一片区域？

它们分别引出向量与方位角、凸包、半平面交。角度有误差时，把前后两类知识接起来，就能得到角楔。直到本课最后的应用习题，我们才处理自己的 Q1。

本课的达标标准：能手算一遍五点凸包；能用叉积判断左右；能把“方向在两个角之间”画成区域并写成两个不等式；能解释一个新不等式怎样裁掉多边形。暂时不需要学三维凸包，也不需要背算法发展史。

建议把本课分为三次：坐标与经典测高例题约 80 分钟；凸包、手算和复做约 100 分钟；半平面、角楔、Q1 应用和代码约 140 分钟。加上画图、错题和口述，可占第一天及第二天前半。时间按自己的计算速度调整。

本课的教学顺序参考 OpenStax 开放教材和 Dave Mount 的计算几何课程讲义；凸包的具体五点数据取自 CGAL 作者维护的入门示例。这样既有标准知识的教材出处，又有可逐项核对的实际例题输入。手算、图和讲解在这里重新展开；没有把自行补充的计算冒称原书答案。

1 坐标是地点，向量是从这里到那里的移动

1.1 先看一对点

在方格纸上标出 $A = (1, 2)$ 、 $B = (4, 6)$ 。这两个数的意思是：先横着走，再竖着走。若把向右、向上定为正方向，从 A 到 B 就要向右走 3、向上走 4。

把这次移动记为

$$\overrightarrow{AB} = B - A = (4 - 1, 6 - 2) = (3, 4).$$

上面是一个最小计算演示，用来认识记号。点是位置；向量是变化。两者都可以写成两个数，但使用时要说清含义。

勾股定理给出这次移动的直线距离：

$$|AB| = \sqrt{3^2 + 4^2} = 5.$$

写成一般公式，就是

$$\|B - A\| = \sqrt{(b_x - a_x)^2 + (b_y - a_y)^2}.$$

双竖线 $\|\cdot\|$ 在本课只表示“向量的长度”。下标 x, y 分别表示横、纵分量。

1.2 把长度拿掉，只保留方向

向量 $(3, 4)$ 与 $(6, 8)$ 指向相同，只是后者长一倍。用向量除以自身长度，可以把箭头缩成长度 1：

$$\frac{(3, 4)}{5} = \left(\frac{3}{5}, \frac{4}{5}\right).$$

这样的箭头叫单位向量。如果方向角为 θ ，它也可以写成

$$u(\theta) = (\cos \theta, \sin \theta).$$

本课从向右的方向开始量角，逆时针为正。因此 $u(0^\circ) = (1, 0)$, $u(90^\circ) = (0, 1)$, $u(180^\circ) = (-1, 0)$ 。把这三个方向亲手画一遍。

若三角函数已经生疏，只需先恢复这件事：在直角三角形里， $\cos \theta$ 是邻边除以斜边， $\sin \theta$ 是对边除以斜边， $\tan \theta$ 是对边除以邻边。把斜边设成 1，就得到上面的单位向量。

1.3 度和弧度是同一个角的两种单位

一整圈既可以记成 360° ，也可以记成 2π 弧度。所以

$$\text{弧度} = \text{度} \times \frac{\pi}{180}.$$

在纸上写 $\sin 30^\circ = 1/2$ 没有问题。Python 的 `math.sin` 接受弧度，因此示例写成：

```
math.sin(math.radians(30))
```

直接写 `math.sin(30)`，参数就代表 30 弧度。

当已知移动向量 $(\Delta x, \Delta y)$ 时，`atan2( $\Delta y$ ,  $\Delta x$ )` 根据两个分量恢复方向。它比直接算 `atan( $\Delta y / \Delta x$ )` 方便：竖直方向不用除以零，左右两个半平面也能区分。

停下来检查。若从 A 到 B 的向量是 $(3, 4)$ ，从 B 到 A 是什么？答案为 $(-3, -4)$ 。长度仍为 5，方向反转。若这个问题还需要猜，先不要往下背公式。

2 经典例题：用两个地面站求飞机高度

2.1 这是什么方法，从哪里来

这是三角测量的基础例题：已知一个三角形的一条边和两个角，恢复其他边长。这里用到的标准工具是正弦定理。

例题数据取自 OpenStax 《Precalculus 2e》第 8.1 节 Example 6，标题为 “Finding an Altitude”。两地面站相距 20 英里，飞机位于两站之间的上空，图中两仰角为 15° 、 35° 。下图按原题数据重新绘制。[G1]

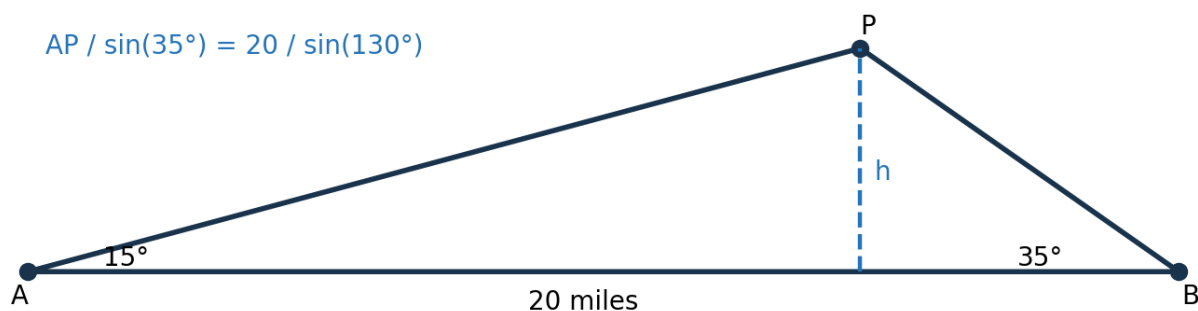


图 1: 两站测高：已知地面基线和两个仰角

图中的平面是竖直剖面。 A, B 是地面站， P 是飞机， h 才是要求的高度。先分清“飞机到地面站的斜距”与“飞机到地面的高度”。

2.2 正弦定理为什么成立

从一个三角形顶点向对边作高。若这条高分别落在两个直角三角形里，可以把同一个高度写成

$$h = b \sin A = a \sin B.$$

把两边除以 ab ，得到

$$\frac{\sin A}{a} = \frac{\sin B}{b}.$$

换另一个顶点作高，同样能接上第三组“角与它正对的边”。于是

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C}.$$

你需要记住的关系是同一个比值里，边必须与角相对。只记公式里的字母，遇到换图就容易错。

2.3 一行一行求出高度

第一步：补出第三个角。三角形三个内角的和为 180° ，所以飞机处的内角为

$$\angle APB = 180^\circ - 15^\circ - 35^\circ = 130^\circ.$$

第二步：找一条斜边。边 AP 对着 35° ，地面边 $AB = 20$ 对着 130° ，所以

$$\frac{AP}{\sin 35^\circ} = \frac{20}{\sin 130^\circ}, \quad AP = \frac{20 \sin 35^\circ}{\sin 130^\circ}.$$

第三步：从斜边取出竖直高度。在左侧直角三角形里， h 是 15° 的对边，因此

$$h = AP \sin 15^\circ = \frac{20 \sin 35^\circ \sin 15^\circ}{\sin 130^\circ} \approx 3.87582 \text{ 英里}.$$

按原书要求保留一位小数，就是 3.9 英里。

这一计算的结构是“从两个精确方向确定交会位置”。它给出一个点的原因，是题里把角度当成精确已知。请先把这个理想情形学会，角度不精确的情形留到第 7 节。

2.4 只用原题做三个检查

1. 为什么不能把 $35^\circ - 15^\circ = 20^\circ$ 当作飞机处的内角？
2. 若先求 BP ，它应该和哪个角放在同一个正弦定理比值里？
3. 最后得到 AP 后，为什么还需要再乘 $\sin 15^\circ$ ？

答案依次为：两站的底边朝向相反，须使用三角形内角和； BP 对着 15° ； AP 是斜距，而要求的是竖直分量。三问都能画图解释，再进入下一种几何问题。

3 凸包：橡皮筋围住一堆点

这部分属于计算几何的入门内容。Dave Mount 在马里兰大学 CMSC 754 第 2 讲中，同样从橡皮筋直观开始，再讲凸性、方向判定和扫描算法；讲义还明确对应 de Berg 等人的《Computational Geometry: Algorithms and Applications》第 1 章。[G5] 本课把必要基础拆得更细，再用 CGAL 的五点输入做完整手算。

3.1 先理解“凸”，再理解“包”

在实心圆盘内部任意选两点，把它们连起来，整条线段都留在圆盘里。实心三角形和实心矩形也有这个性质。这样的集合叫凸集。

反过来，在一个月牙形区域里，某些内部点之间的连线会穿出月牙。这时它就不是凸集。

严格定义只把刚才的话写成公式：若 A, B 都在集合里，那么

$$(1-t)A + tB, \quad 0 \leq t \leq 1,$$

也都在集合里。这里 $t = 0$ 是 A ， $t = 1$ 是 B ，中间的 t 依次走过它们之间的线段。例如 $t = 1/2$ 是中点。

凸包是包含给定全部点的最小凸集。想象每个点是一根钉子，用橡皮筋把所有钉子围住，然后收紧。橡皮筋围起的区域就是凸包。最外面的钉子成为凸包顶点；完全在里面的钉子不改变这条边界。

数学上凸包包括内部。程序通常只保存按边界顺序排列的顶点，这已经足以重建整个凸多边形。

3.2 经典实例：CGAL 的五个点

CGAL 官方 `array_convex_hull_2.cpp` 入门例子给出的五点是

$$A = (0, 0), \quad B = (10, 0), \quad C = (10, 10), \quad D = (6, 5), \quad E = (4, 1).$$

坐标逐项取自原例；字母标签是本课加的。原例调用计算几何库，本课接下来用同一份输入，自己手算 Andrew 单调链。[G2]

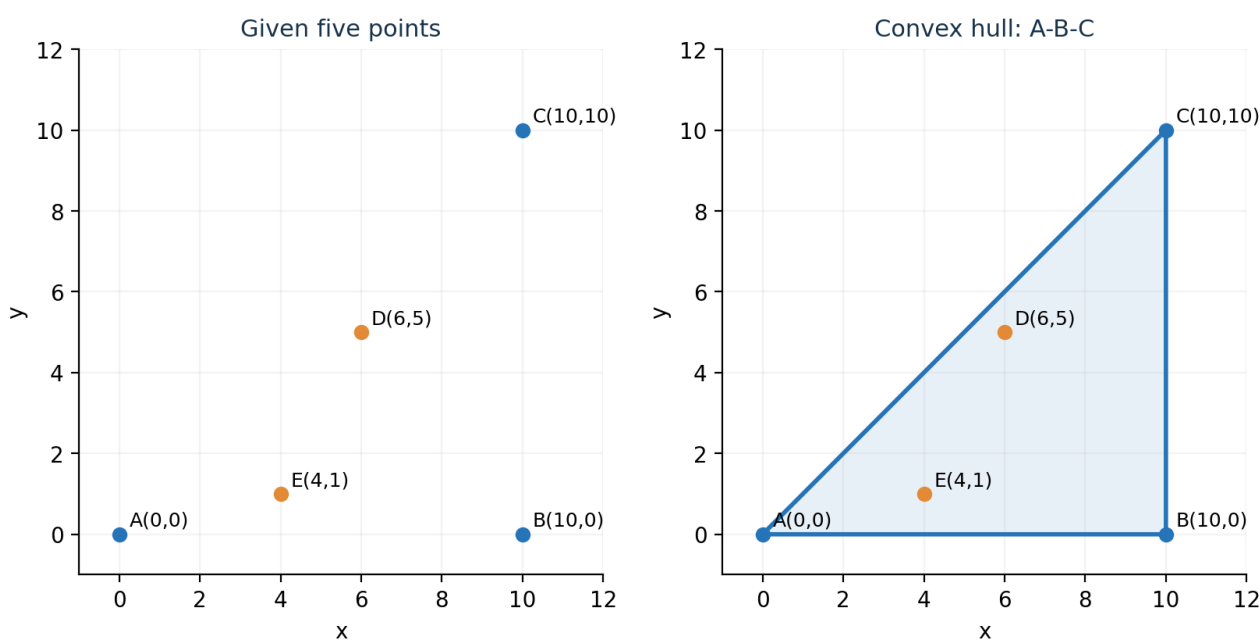


图 2: CGAL 官方五点输入以及它的凸包

先用眼睛判断：橡皮筋应经过 A, B, C ， D, E 被包在里面。现在不要急着看算法，先证明这个观察。

三角形 ABC 中的点恰好满足

$$0 \leq y \leq x \leq 10.$$

检查 $D = (6, 5)$: $0 \leq 5 \leq 6 \leq 10$; 检查 $E = (4, 1)$: $0 \leq 1 \leq 4 \leq 10$ 。所以三角形确实包含全部五点。另一方面，任何包含 A, B, C 的凸集，都必须包含它们之间的整个三角形。于是这个三角形就是所求的最小凸集。

五点例容易看出来。如果有五千个点，就需要可重复执行的步骤。下一节的算法完成的正是这件事。

4 用叉积判断转弯，再用 Andrew 单调链求凸包

4.1 为什么算法要知道左转还是右转

沿着凸多边形边界逆时针走，不能突然向内凹一下。我们需要一个只用坐标就能判断转弯方向的办法。

二维向量 $u = (u_x, u_y)$ 、 $v = (v_x, v_y)$ 的叉积在这里记成一个数：

$$\text{cross}(u, v) = u_x v_y - u_y v_x.$$

在向右为 x 正方向、向上为 y 正方向的坐标系中，给出三个点 A, B, P ，计算

$$c = \text{cross}(B - A, P - A).$$

若 $c > 0$ ， P 在有向直线 $A \rightarrow B$ 左侧；若 $c < 0$ ，在右侧；若 $c = 0$ ，三点共线。

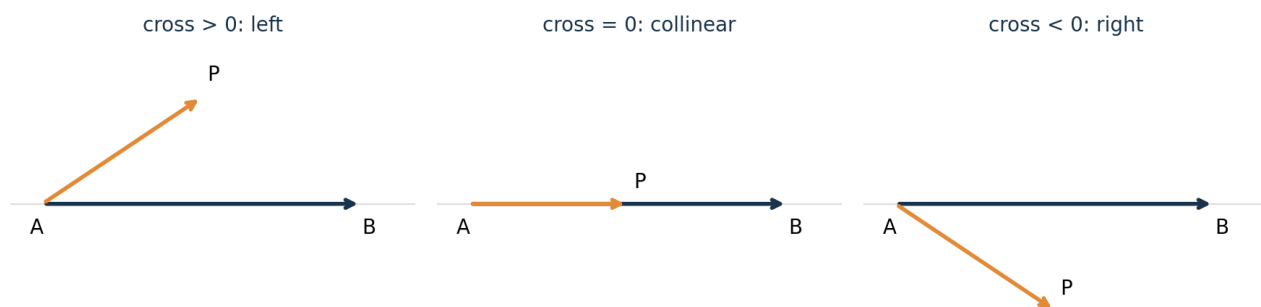


图 3: 叉积的符号把左侧、共线、右侧分开

这个符号有面积意义：

$$\text{三角形 } ABP \text{ 的面积} = \frac{|c|}{2}.$$

绝对值给面积大小，正负号保留左右顺序。因此这里可以不算角度、不调用反三角函数，就判断转弯。

4.2 先拿经典五点例中的三个点手算

用 $A = (0, 0)$ 、 $E = (4, 1)$ 、 $D = (6, 5)$ ：

$$\text{cross}(E - A, D - A) = \text{cross}((4, 1), (6, 5)) = 4 \times 5 - 1 \times 6 = 14 > 0.$$

所以 $A \rightarrow E \rightarrow D$ 是左转。

代码也常写成 $\text{cross}(E - A, D - E)$ 。它仍等于 14，因为

$$\text{cross}(E - A, D - E) = \text{cross}(E - A, D - A) - \text{cross}(E - A, E - A),$$

最后一项为零。两个公式使用了不同起点，但判断同一个转弯。这能解释源码为什么把第二个向量写成“新点减上一个点”。

4.3 标准算法的名字与步骤

这里教的是 Andrew 单调链凸包算法，也常叫 Graham–Andrew 扫描的一种形式。CGAL 作者文档介绍了该算法及其 $O(n \log n)$ 时间复杂度。[G3] Mount 讲义第 4–8 页讲的是按横坐标排序、维护上下链的扫描变体；它与本课使用的转弯检查、删除旧拐角属于同一条教学路线。[G5]

“单调”说的是：先按横坐标从小到大走一遍，再反方向走一遍。算法只做四件事：

1. 删除重复点，按 x 从小到大排序； x 相同就比较 y 。
2. 正着扫，维护下边界。每来一个点，检查末尾三点是否仍然左转。
3. 若不是左转，删掉中间那个旧点，继续向前检查；合格后再加入新点。
4. 反着扫得到上边界，把两条链拼起来，公共端点只保留一次。

“删掉旧点”在程序里叫 `pop`。这里删除的是当前链末尾不合格的拐角；并不是把点从原始输入里永久抹掉。正向链没有使用的点，仍可能出现在反向链中。

4.4 经典五点例：完整计算下边界

排序结果是

$$A(0,0), E(4,1), D(6,5), B(10,0), C(10,10).$$

开始时列表为空。少于两个旧点就直接加入，因为还凑不成一个转弯。

读入点	需要检查的转弯	叉积	处理后列表
A	不足三个点	—	A
E	不足三个点	—	A, E
D	$A \rightarrow E \rightarrow D$	14	A, E, D
B	$E \rightarrow D \rightarrow B$	−26	先删 D
仍是 B	$A \rightarrow E \rightarrow B$	−10	再删 E ，加入 B : A, B
C	$A \rightarrow B \rightarrow C$	100	A, B, C

其中第一次删除的算式为

$$\text{cross}(D - E, B - D) = \text{cross}((2, 4), (4, -5)) = 2(-5) - 4(4) = -26.$$

删掉 D 后必须再次检查。因为新的末尾三点已经变成 A, E, B ，它也向错误一侧转弯：

$$\text{cross}(E - A, B - E) = \text{cross}((4, 1), (6, -1)) = -10.$$

这就是算法使用 `while` 循环的原因：一个新点可能让多个旧拐角失效，单次 `if` 不够。

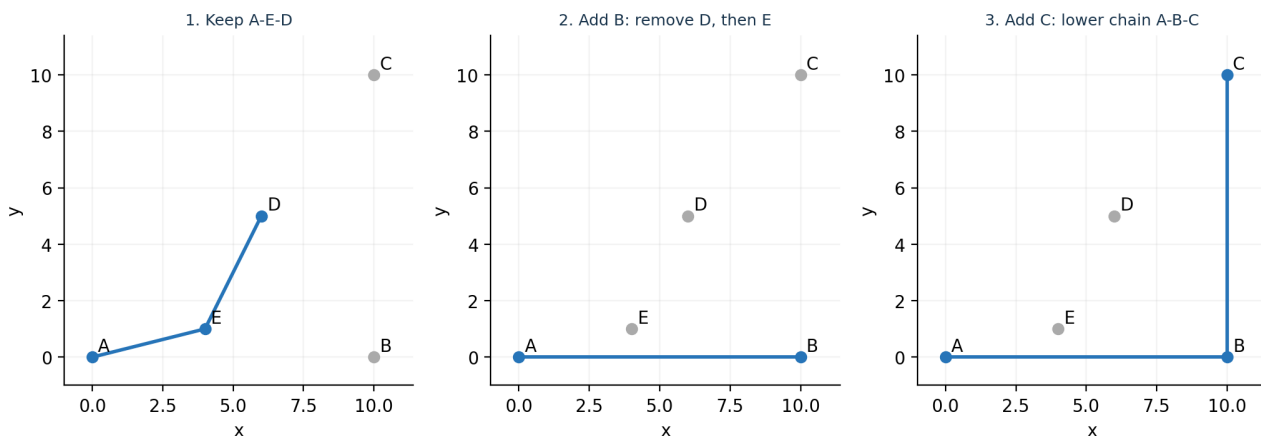


图 4: 同一组五点: 新点 B 到来时, 下边界连续删除两个旧拐角

4.5 上边界也要亲自算一遍

反向输入为 C, B, D, E, A 。前三点 $C \rightarrow B \rightarrow D$ 的叉积是 -40 , 删 B , 留下 C, D 。

加入 E 时, $C \rightarrow D \rightarrow E$ 的叉积为 6 , 暂时得到 C, D, E 。加入 A 时:

1. $D \rightarrow E \rightarrow A$ 的叉积为 -14 , 删 E ;
2. $C \rightarrow D \rightarrow A$ 的叉积为 -10 , 再删 D ;
3. 上边界最后为 C, A 。

下边界 A, B, C 和上边界 C, A 各去掉最后一个点, 再拼接, 得到 A, B, C 。这样公共端点 A, C 都只出现一次。

4.6 它为什么快, 什么时候要特别处理

排序需要 $O(n \log n)$ 时间。扫描中每个点最多入链一次、出链一次; 看起来有两层循环, 累计工作却是 $O(n)$ 。所以总量级为 $O(n \log n)$ 。

正确性的直观依据是: 在已经排好序的点中, 一个向内折回的拐角不可能属于正在构造的外侧边界; 去掉它后继续检查, 更早的边界仍按同一规则维护。上、下两条外侧边界合起来围住全部输入点。

共线的点要先约定输出形式。若只保留凸包拐角, 就把叉积等于零时的中间点也删除。若希望边上的每个输入点都保留, 就需要调整规则。几何区域可以相同, 返回的点列表却不同。

还应处理零点、一点、两点; 它们的凸包分别为空、一个点、一条线段, 不能强行当成有面积的多边形。

4.7 本节必须会的复做

盖住第 4.4、4.5 节, 只看 CGAL 原始五点, 重新写出排序、下链、上链。检查自己是否能解释以下三句:

- 为什么必须先排序? 因为维护上、下边界时, 需要新点按一个方向有序到来。
- 为什么有时连续删两个点? 因为删一次以后, 新的末尾转弯也可能不合格。
- 为什么凸包只有三个顶点? 因为另外两点在这个三角形内部。

能独立复做这个例题, 就达到了本课对凸包的要求。无需现在背会其他凸包算法。

5 半平面交：同时满足几个条件

5.1 从一个不等式开始

直线 $x = 10$ 把平面分为两边。条件 $x \leq 10$ 选择它左侧所有点，并包含直线本身；这叫一个闭半平面。

一般形式为

$$ax + by \leq c.$$

(a, b) 是垂直于边界的方向，叫法向量。这里可以先用最直接的办法理解它：把一个点的坐标代入左边，若结果不大于右边，该点就被保留。

同一个半平面也写为

$$n \cdot X \leq c, \quad n = (a, b), \quad X = (x, y), \quad n \cdot X = ax + by.$$

中间点号是点积，在二维就是“对应分量相乘后相加”。不要把它和上一节的叉积混淆：一个是加法，一个是交叉相乘再相减。

5.2 仍然使用刚才的经典三角形

CGAL 五点的凸包三角形可以写成三个半平面的共同部分：

$$y \geq 0, \quad x \leq 10, \quad y \leq x.$$

把它们统一写成“左边小于等于右边”：

$$-y \leq 0, \quad x \leq 10, \quad -x + y \leq 0.$$

一个点要同时通过三项检查。满足第一个而不满足第二个，仍然不能留下。

“同时满足”对应集合的交，符号是 $\cap$ 。若三个半平面分别为 H_1, H_2, H_3 ，三角形就是 $H_1 \cap H_2 \cap H_3$ 。求这个交集的几何问题叫半平面交。

此处是对同一经典数据的独立延伸计算：原 CGAL 示例输入点，本节把已经求出的三角形改写成不等式，没有说原示例还执行了这些步骤。

5.3 为什么交出来的区域仍然是凸的

先看一个半平面。若 A, B 都满足 $n \cdot X \leq c$ ，它们之间的点为 $X = (1 - t)A + tB$ ，则

$$n \cdot X = (1 - t)(n \cdot A) + t(n \cdot B) \leq (1 - t)c + tc = c.$$

所以整条线段都在半平面内。若 A, B 同时在多个半平面里，这个证明对每一条约束都成立，线段也就同时在每一个半平面里。

因此，多个半平面的交一定是凸集。它可能是多边形，也可能为空、无界、一个点或一条线段。

请区分两种输入：凸包从点出发，找包住全部点的边界；半平面交从不等式出发，找满足全部条件的区域。它们经常得到同一个凸多边形，却解决不同的输入问题。

6 怎样真的算交集：一次只裁掉一侧

6.1 标准方法：Sutherland–Hodgman 多边形裁剪

这是一种计算机图形学里的经典算法：一个多边形有一部分跑到显示窗口之外，要把外部部分裁掉。赫尔辛基大学公开教学页用箭头形多边形演示了按窗口左、上、右、下边界逐次裁剪，以及四种进出边界的情况。[G4]

本课先看懂一条直线对应的一次裁剪。只要会做一次，再把新的多边形交给下一条直线，重复即可。

已有多边形的顶点必须按绕边界的顺序存放。沿边 $A \rightarrow B$ 移动，起点和终点各有“内、外”两种状态，于是恰好四种情况：

起点 A	终点 B	应往结果列表加入什么	直观含义
内	内	B	整条边保留
内	外	交点 I	走到边界就停止
外	内	交点 I ，再加 B	从边界进入，再走到终点
外	外	不加点	整条边留在这一半平面的外侧

这张表就是原教学页的四种情形，用中文重新说明。这里每次裁的是一个半平面；“两端都在外面”才可以直接删除整条线段。不要把该规则误用于任意弯曲窗口。

6.2 边界上的交点怎么求

设要保留 $n \cdot X \leq c$ ，定义一个方便检查的数

$$f(X) = n \cdot X - c.$$

$f(X) \leq 0$ 是内部， $f(X) > 0$ 是外部。边上的点可写成

$$I = A + t(B - A).$$

交点在边界上，所以 $f(I) = 0$ 。把上式代入：

$$0 = f(A) + t(f(B) - f(A)), \quad t = \frac{f(A)}{f(A) - f(B)}.$$

两端真正处在不同侧时， t 在 0 到 1 之间。这也是计算结果的自检条件。这里没有斜率，水平边和竖直边都可以使用。

6.3 读图练习：四种情况在一张图里

下面的图是为练四种情况绘制的数值演算图，并非原大学教学页的坐标。虚线矩形是初始区域，橙线为 $x + 0.4y = 2.8$ ，保留右侧蓝色部分。

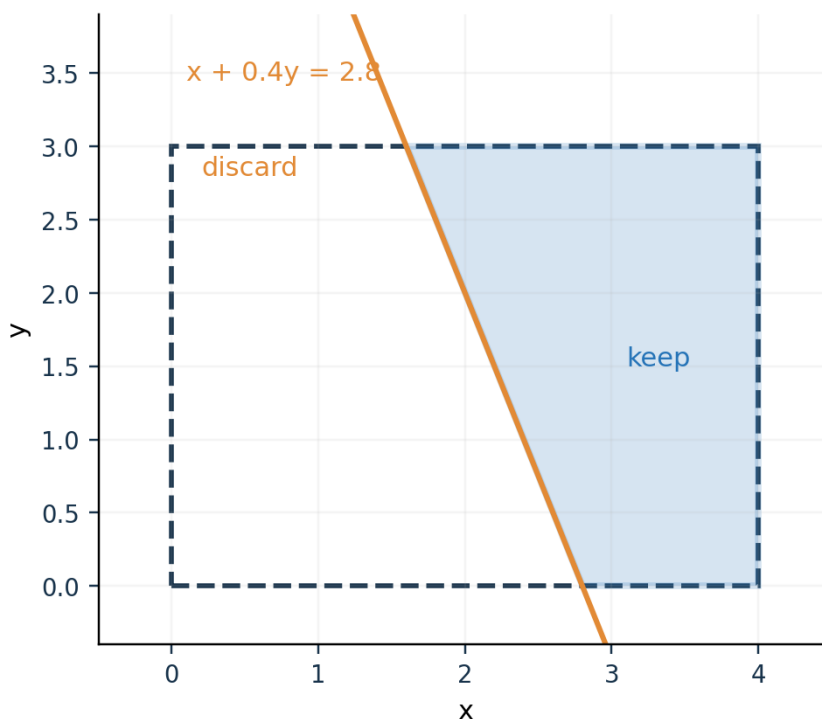


图 5: 逐边裁剪的四种情况：蓝区和橙线使用同一个不等式

从 $(0,0)$ 起逆时针绕矩形：

- 下边：外到内，输出 $(2.8, 0)$ 、 $(4, 0)$ ；
- 右边：内到内，输出 $(4, 3)$ ；
- 上边：内到外，输出 $(1.6, 3)$ ；
- 左边：外到外，不输出。

因此新多边形的四个顶点为

$$(2.8, 0), (4, 0), (4, 3), (1.6, 3).$$

验证交点并不难： $2.8 + 0.4 \times 0 = 2.8$ ， $1.6 + 0.4 \times 3 = 2.8$ 。两点确实在同一条边界上。

6.4 另一种实现：把边界排好序，用双端队列维护

逐边裁剪需要一个已有的有界多边形。有些程序直接收到一组半平面，不预先给一个大矩形，就会使用按边界方向排序的半平面交。

它的工作对象换成了边界列表：

1. 给每条边界规定行走方向，使保留的一侧总在左边；
2. 按方向角排序；同向平行边界只保留更严格的一条；
3. 把可能成为最终边界的直线依次放进一个双端队列；
4. 加入新边界时，检查队列两端相邻边界的交点；若交点被新条件排除，就移走相应的旧边界，再检查；
5. 首尾也做同样清理，最后求相邻边界交点。

双端队列可以从头、尾两边取走元素。它节省的是“每次加入新条件都从头重算”的工作。这与上一节凸包里的删除无效拐角有相似之处，但队列此时存的是半平面，不是坐标点。

用前面三角形作一次独立延伸：在 $y \geq 0, x \leq 10, y \leq x$ 之外，加入多余条件 $x + y \leq 30$ 。暂时由 $x = 10, x + y = 30$ 得到交点 $(10, 20)$ ；检查新条件 $y \leq x$ ，它不合格。清理后这条多余斜边不再成为最终边界。三角形仍为 ABC 。

本课对这一实现的要求：会说队列里存什么、为什么检查交点、为何需要同向合并、为何不能把无界区域误当多边形。完整队列退化处理放在最后的源码导读中，不要求第一遍阅读就背下全部分支。

7 方向读数有误差，射线就扩成角楔

7.1 先用普通测量语言说清楚

前面的飞机例题把一个方向视为确定的。现在想象一个平面方向仪报告“ 30° ，误差不超过 10° ”：真实方向可能是 20° ，也可能是 34° ，还可能是 40° 。报告没有告诉你距离。

所以应保留从 20° 到 40° 之间的所有前向射线。这块向外张开的区域叫角楔。这里使用 10° 是为了图上看得清，属于一般误差几何示意。

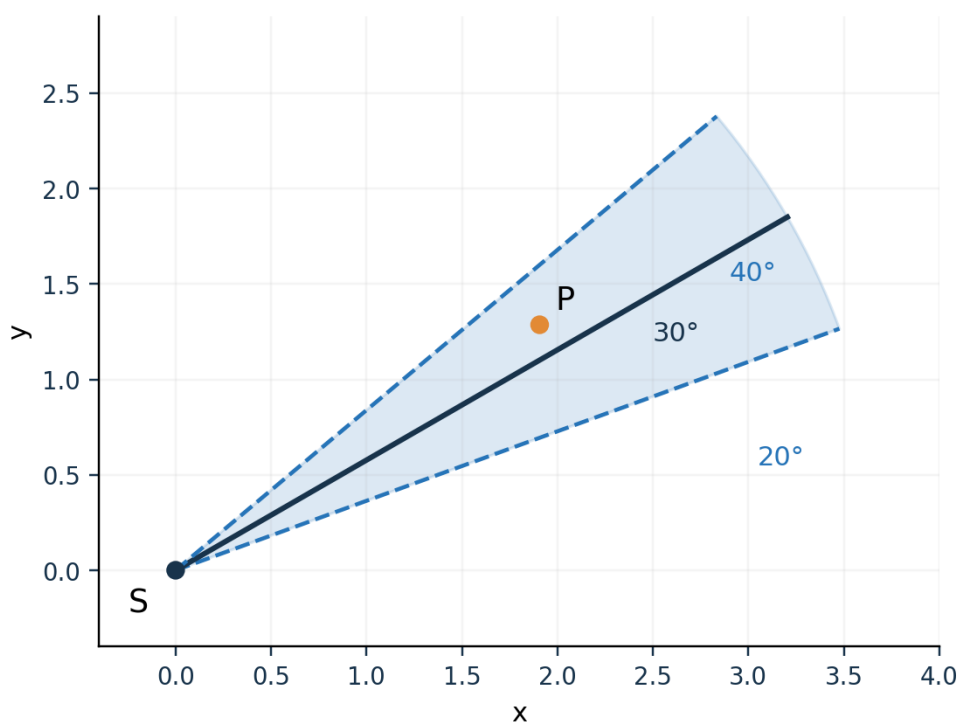


图 6: 报告方向是中间线，允许方向形成两条边界之间的角楔

图里的蓝色只画到有限距离，方便显示；若没有额外距离信息，数学上的角楔一直向外延伸。它不是一个有限扇形，也不是一个概率分布。

7.2 用已经学会的“左右侧”写下来

设测量位置为 S ，报告方向为 θ ，误差半宽为 δ 。令

$$u_- = u(\theta - \delta), \quad u_+ = u(\theta + \delta), \quad v = X - S.$$

先只考虑 $0 < \delta < 90^\circ$ ，所以整块楔形张角小于 180° 。一个候选位置 X 要在角楔里，必须满足两句话：

1. v 在下边界 u_- 的左侧或边界上；
2. v 在上边界 u_+ 的右侧或边界上。

用叉积翻译，就是

$$\text{cross}(u_-, v) \geq 0, \quad \text{cross}(u_+, v) \leq 0.$$

你不必先背两个符号。画一条向右的下边界，在它上方选点，叉积应为正；画上边界，在它下方选点，叉积应为负。这样可以自己恢复公式。

7.3 把它变成裁剪程序接受的格式

程序接受 $n \cdot X \leq c$ ，所以把两个叉积展开。

下边界的条件为

$$\cos(\theta - \delta)(y - s_y) - \sin(\theta - \delta)(x - s_x) \geq 0.$$

两边乘以 -1 ，再把与 S 有关的常数移到右侧，得到

$$n_- \cdot X \leq n_- \cdot S, \quad n_- = (\sin(\theta - \delta), -\cos(\theta - \delta)).$$

同样展开上边界：

$$n_+ \cdot X \leq n_+ \cdot S, \quad n_+ = (-\sin(\theta + \delta), \cos(\theta + \delta)).$$

现在你已经从图独立推到了两个半平面。一次读数给两条约束，几次读数就把这些约束全部放在一起求交。

7.4 两个容易忽略的边界情况

跨过 0° 。例如报告接近 359° ，真实方向接近 0° ，它们可以只差一度。若直接相减，会误以为相差几百度。向量写法通过正弦、余弦的周期性自然跨过零度；角差也可归到 $[-180^\circ, 180^\circ)$ 再比较。

误差恰为零。当 $\delta = 0$ ，两条边界重合，两个半平面只限制出一条完整直线。还需补上

$$u(\theta) \cdot (X - S) \geq 0,$$

把背后的半条线去掉，才能得到正向射线。

7.5 为什么同样的角度误差，远处位置更不确定

把坐标轴暂时转到报告方向：沿报告方向的距离记为 $a \geq 0$ ，垂直方向的偏移记为 b 。看一个直角三角形，便有

$$|b| \leq a \tan \delta.$$

所以在前向距离 a 处，楔形横截面的全宽为 $2a \tan \delta$ 。距离翻倍，全宽也翻倍。这里比较的是同一个角度误差，并没有把它误写成固定的坐标误差。

8 课后应用：把 Q1 当作本课综合习题

到这里，你已经学习了通用方法。下面才接回自己的题：多个检测点给出平面示向度，每条读数允许 $\pm 1^\circ$ 的角误差，求允许位置区域，并进一步描述它的大小。直径与包围圆放到下一课。

习题 Q1-A：先独立建模，不看代码

检测点为 S_i ，报告为 θ_i 。请完成四步：

1. 画第 i 条读数的两条边界；
2. 写出它对应的两个半平面；
3. 写出全部读数共同允许的集合；
4. 解释为什么不能直接把报告中心线的交点当成完整答案。

参考解答。令 $\delta = 1^\circ$ ，每个角楔为

$$W_i = \{X : n_{i,-} \cdot X \leq n_{i,-} \cdot S_i, n_{i,+} \cdot X \leq n_{i,+} \cdot S_i\}.$$

仅使用这些角观测得到

$$F_{\text{angle}} = W_1 \cap W_2 \cap \cdots \cap W_m.$$

中心线相当于擅自把误差设为零。即使它们恰好交于一点，也没有代表全部允许位置；若带误差的中心线不共点，仍可能存在非空的共同允许区域。

与经典飞机例题相比，几何空间从竖直剖面换成水平面，已知方向从精确角换成区间，输出从一个位置换成一片位置集合。三角函数与方向关系仍可用，求解目标发生了变化。

习题 Q1-B: 用真实输入检查第一条约束

打开仓库中的 `examples/q1_seed_20260929_input.json`。第一站为

$$S_1 = (-1387.941092, 283.869424), \quad \theta_1 = 51.26^\circ.$$

因此两条边界角为 50.26° 、 52.26° 。把它们代入第 7.3 节，得到下列近似值：

$$\begin{aligned} n_- &= (0.768953424, -0.639304804), \quad n_- \cdot S_1 \approx -1248.741141, \\ n_+ &= (-0.790796414, 0.612079269), \quad n_+ \cdot S_1 \approx 1271.329428. \end{aligned}$$

纸笔检查两件事：代入 $X = S_1$ ，两条不等式都恰好取等号；代入从 S_1 沿报告方向向前走的点，两条约束都应通过。显示值经过舍入，精确边界判断以程序保留的浮点数为准。

习题 Q1-C: 逐条加入读数，预判形状怎样变

用同一个固定输入，依次只使用前 1、2、3、4 条观测。先预测：新增条件后，旧区域中的点会增加还是减少？区域直径一定严格减小吗？

本仓库现有求解器复算结果如下，单位为米：

使用观测数	状态	顶点数	直径
1	unbounded 无界	不返回有限多边形	无有限值
2	polygon	4	49.427379
3	polygon	4	44.458564
4	polygon	6	44.458564

解释：取交不会加入原来没有的点，所以理想精确模型下区域只会缩小或保持。直径可能不变，最后两行就是例子。顶点数反而可以增加，因为一条新边界可能切掉一个旧角并产生两个新角；顶点更多不表示区域更大。

这些是现有固定种子输入的本地复算，作用是练读程序。不要把四行数据说成一般定理；“三条观测一定够”“四条一定更好”都推不出来。

习题 Q1-D: 目标圆域有没有真的放进程序

题目还有目标所在区域。若把半径 1800 米的圆盘记为 Ω ，同时使用这个先验与角观测时，应写成

$$F = \Omega \cap F_{\text{angle}}.$$

但当前 Q1 求解器的 `intersect_halfplanes` 第 203 行直接删除了 `include_circle` 参数；其计算结果是角约束的半平面交，并没有加入目标圆盘约束。生成器把测试真值放在圆内，也不等于求解器已经使用圆域裁剪。

这道练习要培养的能力是：从输入和真正执行的代码确认结论，而不是根据参数名字猜功能。答辩解释现有版本时，准确说“这里计算多条角度观测的共同允许区域”即可。若被问到圆域，再说明现有函数的范围。

9 最后才打开代码：把学过的动作逐项找出来

本节行号对应仓库当前 `source/Q1/q1_solver.py`。阅读顺序按调用含义安排；不要从第一行 `import` 开始背。

9.1 先读入口，确定程序拿了什么

`main` 在第 283–298 行。第 288 行读 JSON，第 289 行只把 `observations` 和 `epsilon_deg` 交给 `solve_localization`。生成器附带的 `local_truth` 没有作为定位输入。

`solve_localization` 在第 268–280 行：

行号	做的事	本课对应的数学动作
269–271	检查误差半宽属于 $[0, 90^\circ)$	保证用两个半平面表示前向小角楔的模型范围
272	规范化各条观测	提取站点坐标和方向角
273–274	拒绝空观测输入	没有读数时不假装已经定位
275	每条观测调用 <code>_wedge</code> ，收集全部约束	一条读数给两个半平面；零误差另有正向约束
276	调用 <code>intersect_halfplanes</code>	所有约束取交
277–280	附上输入说明并返回	方便结果追溯

9.2 再读 `_wedge`：现在每一个符号都有来处

第 257–265 行依次完成：

1. 取出测站 $S = (x, y)$ ；
2. 计算 $\theta - \delta, \theta + \delta$ ，并转为弧度；
3. 建立第 7.3 节推出来的 n_-, n_+ ；
4. 若误差为零，补上正向限制；
5. 返回 `HalfPlane(normal, normal · S)`。

`HalfPlane` 在第 20–27 行保存 $n \cdot X \leq c$ 。`residual` 返回 $n \cdot X - c$ ，正好是裁剪一节的 $f(X)$ 。它的正负告诉你一个点在哪边。

9.3 凸包代码：把五点手算逐行贴回去

凸包函数 `_convex_hull` 在第 144–157 行。

行号	Python 写法的作用	经典五点例中发生什么
145	转浮点、去重、排序	得到 A, E, D, B, C
146–147	零点或一点直接返回	无需构造链
149–150	定义扫描过程并建立空列表	准备存当前边界
151	按给定顺序取点	依次读 A, E, D, B, C
152	检查末尾三点叉积是否不大于容差	读 B 时先算 -26 ，再算 -10
153	删除旧的末尾点	先删 D ，再删 E
154	加入新点	加入 B ，继续到 C
157	正、反扫，去掉重复端点后连接	得到 A, B, C

`result[-1]` 是列表最后一个点，`result[-2]` 是倒数第二个点。`[:-1]` 表示保留除最后一个之外的元素；`reversed(rows)` 表示反向读取。

理论上用叉积 ≤ 0 删除非左转点，源码使用 $\leq \text{EPS}$ 。 $\text{EPS}=1\text{e-}10$ 是处理浮点近零值的实现容差。它不是物理上的 1° 测量误差，两者的含义不能混用。

9.4 半平面交代码：先会讲主干，再查异常分支

`intersect_halfplanes` 在第 201–237 行，使用第 6.4 节的方向排序与双端队列；本 Q1 没有调用逐边裁剪函数。

函数或行号	核心作用	为什么需要
<code>_normalise</code> , 43–59	把法向量长度化为 1	使同向约束的右端值可以直接比较
<code>_ordered</code> , 73–87	按边界方向排序，合并同向约束	给队列提供有序边界；移除较松的同向重复约束
<code>_intersection</code> , 90–97	联立两条边界直线	找出相邻边界候选顶点
<code>_outside</code> , 100–102	检查约束残差并加尺度容差	避免微小浮点误差把边界点误当外点
205–215	队列两端与首尾清理	移除被后续条件排除的旧边界
217–226	求交点，整理凸包，逐约束核验	候选图形必须确实满足全部输入条件
<code>_feasible_point</code> , 110–132	尝试找到一个可行点	用来区分空集和仍有可行点的情形
<code>_is_bounded</code> , 135–141	检查约束方向能否封闭区域	有交点不代表整个集合有界
228–237	返回 <code>empty</code> 、 <code>unbounded</code> 或 <code>uncertain</code>	无法得到可靠有限图形时不伪造数值

归一化的理由可以用一个一眼能算的等价式说明： $x \leq 10$ 和 $2x \leq 20$ 是同一约束。若不先归一化，只拿 10 与 20 比较，会误以为第一条更严格。

两直线平行时没有唯一交点。`_intersection` 通过行列式判断近乎平行并返回 `None`，不拿一个虚构分母继续算。

第 6.4 节的排序与队列主干通常按 $O(m \log m)$ 讨论；当前完整函数还包含全约束核验和可行性补救，不能把主干复杂度不加说明地当作所有分支的复杂度。

9.5 逐边裁剪去哪里找

本课先讲裁剪，是因为它便于画图手算，也会在后续题目反复使用。实际位置是 `source/Q2/q2_solver.py` 的 `clip_halfplane` 第 124–144 行，以及 `clip_wedge` 第 157–166 行。

因此，对代码的准确说法是：Q1 使用排序与双端队列求半平面交；Q2 等处在已有多边形上逐条裁剪。二者都在求共同满足约束的区域，存储对象和更新步骤不同。

10 运行与口头验收

10.1 只运行已有固定输入

在仓库根目录运行：

```
python source/Q1/q1_solver.py --input examples/q1_seed_20260929_input.json --output q1_course_
```

预期状态为 `polygon`，顶点数为 6，面积约 773.517 m^2 ，直径约 44.459 m。下一课再解释直径是怎样计算的。运行后把结果文件与 `examples/q1_seed_20260929_result.json` 对照即可。

10.2 闭卷做一个三分钟说明

请按这四句话的次序讲，每句话后面都能在纸上画出相应对象：

方向用单位向量表示。二维叉积的符号告诉我，一个候选点在有向边界的左边还是右边。
一次有界角误差观测给出一个前向角楔，角楔可以写成两个半平面的交。把全部观测约束一起求交，就得到共同允许区域；程序再根据是否为空、是否有界等情况返回相应结果。

然后打开 `_wedge`，从图推出两行 `normals`，最后打开 `_convex_hull`，用 CGAL 五点例解释一次连续 `pop`。能做到这一点，已经是在理解代码。

10.3 本课评委追问

问：你们为什么不用最小二乘直接求一个点？

答：本段程序的目标是保留满足每条硬误差上界的全部位置。最小二乘通常最小化一组残差的平方和，主要返回一个拟合点；即使能拟合，也还需另外证明整个允许区域或最大偏差。这里用集合交，正好直接表达“每条读数都要满足”。这不表示最小二乘本身错误，而是当前需要的输出不同。

问：半平面交为什么是凸多边形？

答：每个半平面是凸集，交仍然凸；当交集有界、非空且有面积时，直线边界围出凸多边形。也可能出现空、无界、点和线段，因此程序要分类。

问：你们是不是先把检测点做凸包，就得到源的位置？

答：不是。定位约束由检测点和方向读数共同生成，先求半平面交；凸包用于整理候选顶点及后续几何计算。检测点的凸包本身不能表达示向角条件。

问：为什么不直接比较两个角的大小？

答：角度有 360° 周期，零度两边的方向可能很近。用正弦、余弦和叉积描述两条边界，周期性自然保留；输入角也先规范化。

问：新观测一定让定位精度变好吗？

答：在观测可靠、约束正确的模型下，取交不会扩大集合；但可能新增的是冗余条件，所以某个大小指标未必严格下降。固定输入中，从三条到四条观测的直径就保持不变。

问：为什么浮点容差不能完全设成零？

答：三角函数和交点计算使用浮点数，理论在边界上的点可能出现极小正残差。容差用于处理这类数值误差；它应该受尺度和模型要求控制，不能随意放大后冒充原始物理误差。

本课材料出处与阅读范围

- [G1] OpenStax, Precalculus 2e, § 8.1, Example 6, “Finding an Altitude”. 本课采用其 20 英里、 15° 、 35° 测高问题并重新推导。初学者只需读这一例与本节正弦定理，不必一次读完全部三角形分类。<https://openstax.org/books/precalculus-2e/pages/8-1-non-right-triangles-law-of-sines>
- [G2] CGAL 5.6.2, `Convex_hull_2/array_convex_hull_2.cpp`. 五点输入的直接出处。原程序调用 `convex_hull_2`；本课的 Andrew 逐步手算是对相同输入的独立展开，不声称原默认调用必然选择 Andrew。https://doc.cgal.org/5.6.2/Convex_hull_2/Convex_hull_2_2array_convex_hull_2_8cpp-example.html
- [G3] CGAL 5.6.2, 2D Convex Hulls and Extreme Points, User Manual. 参看 “Convex Hull” “Example using Graham-Andrew’s Algorithm” “Extreme Points and Hull Subsequences”。用来核查算法名称、扫描形式与复杂度；无需学习本页的 C++ 类型系统。https://doc.cgal.org/5.6.2/Convex_hull_2/index.html
- [G4] University of Helsinki, “Sutherland–Hodgman Polygon Clipping”, 1996. 重点看逐窗口边裁剪图及 “Four Types of Edges”。本课四种输出规则与该教学页对应，坐标演算图为独立教学绘图。<https://www.cs.helsinki.fi/group/goa/viewing/leikkaus/intro2.html>
- [G5] Dave Mount, University of Maryland, CMSC 754, Lecture 2: “Convex Hulls in the Plane”, Fall 2023. 讲义第 1–3 页给出直观与基本定义，第 4–8 页展开扫描、方向判定与正确性。按这两个范围查阅即可，后面的分治凸包属于拓展。<https://www.cs.umd.edu/class/fall2023/cmsc754/Lects/lect02-hulls.pdf>
- 原始算法文献线索：Ivan E. Sutherland 与 Gary W. Hodgman, “Reentrant Polygon Clipping”, Communications of the ACM, 17(1), 32–42, 1974, DOI: <https://doi.org/10.1145/360767.360802>。这是继续追溯方法的文献，不要求本周通读。
- 本题可运行的一手材料：`source/Q1/q1_solver.py`、`source/Q1/q1_generator.py`、`source/Q2/q2_solver.py`；固定输入与输出位于 `examples/`。本课所列行号与数值按这些文件核查。

进入下一课前：先不看提示复做 CGAL 五点凸包，再把 Q1 第一条读数画成角楔。如果仍说不清“点集凸包”和“约束交集”的区别，停下来画两幅图，暂缓背旋转卡壳和最小包围圆。

第 3 课 直径与最小包围圆

本章分两次学。第 3 课学“给定一块区域，怎样量它”；第 4 课学“有几种可能结果时，怎样选一个方案”。学完通用例题之后，再做课末的 B 题应用。前面的消防站、凸多边形与函数最小值，都不需要知道比赛背景。

进入本章前，你只需会三件事：用勾股定理算两点距离，识别凸多边形，会算二维叉积。若叉积仍不熟，先回上一章做一遍“三左左转还是右转”的例题。

先认清名称：我们究竟在学什么

名称	它是什么	用一句日常话说它解决什么
点集或区域的直径	一个几何问题	里面相距最远的两个位置，离多远？
旋转卡壳	求解一类凸几何问题的算法思想	沿凸形外缘转动两条平行支撑线，顺序找候选点对
最小包围圆，MEC	一个几何优化问题	把所有点包住，圆能有多小？
随机增量法	一种求解算法	一个点一个点加入；只有新点破坏当前答案时才修正
极小极大，minimax	一种选择方案的准则	先看每个方案最差会怎样，再挑最差表现最好的
黄金分割搜索	一维数值优化算法	在具有单谷形状的区间上，只比函数值就不断缩小最小值所在区间

这些名称不能互相替代。“我们用了最小包围圆”说明求什么；“我们用随机增量法计算它”才说明怎样求。“鲁棒”在这里指考虑允许范围内的所有情况，不是泛指“程序很稳定”。

3.1 第一种任务：找到相隔最远的一对点

先在纸上画一个凸多边形。你想给它量一个尺寸：拿两个点分别放在区域内，拉开，能拉到的最大距离就是区域直径。

圆的直径恰好是大家熟悉的“两倍半径”。但一般区域也有直径；三角形、四边形、线段都可以谈直径，并不要求它长得像圆。

若区域记作 P ，写成公式是

$$D(P) = \max_{x, y \in P} \|x - y\|.$$

逐项读它：从 P 里选两个位置 x, y ，计算距离，取所有可能选择中的最大值。式子里没有圆心，也没有“覆盖全部位置”的要求。

经典问题出处：Toussaint 的凸多边形直径图例

Godfried Toussaint 在 1983 年论文 Solving Geometric Problems with the Rotating Calipers 的第 1 节、图 1 中，讲的是一个已按边界排好顶点的凸多边形：用两条平行支撑线夹住它，转动支撑线，找最远点对。这就是本节跟学的经典图例。原论文，第 1-2 页

作者的说明页交代了来源：Shamos 于 1978 年用这一思想求凸多边形直径，Toussaint 后来命名并推广了“旋转卡壳”。它属于计算几何，这里是平面算法。作者说明页

原图没有供逐步计算的坐标。因此下面先讲原图表达的操作，再明确用一组教学补充坐标练习；不把补充坐标称为论文原数据。

3.2 为什么不必检查区域内部的每个点

假设多边形有 n 个顶点。即使只检查顶点，两两比较也有 $n(n-1)/2$ 对。区域内部则有无穷多个点。第一步必须解释：为什么能从连续区域退到有限顶点？

直观上，把内部点向某个远离另一点的方向推，总能推到边界上而不使距离变小；沿着一条边继续看，最远位置又能在端点取得。因此最远点对可以选在顶点中。

下面是一段可用于答辩的证明。先用两个边界顶点说明：若 $x = \lambda v_1 + (1 - \lambda)v_2$ ，其中 $0 \leq \lambda \leq 1$ ，那么对固定的 y ，

$$\|x - y\| \leq \lambda \|v_1 - y\| + (1 - \lambda) \|v_2 - y\| \leq \max\{\|v_1 - y\|, \|v_2 - y\|\}.$$

第一步是三角不等式；第二步说加权平均不超过最大项。凸多边形中任意点都是顶点的凸组合，应用同一个道理，先把 x 换成某个顶点，再把 y 换成某个顶点，就证明了

$$D(P) = \max_{i, j} \|v_i - v_j\|.$$

本节掌握线：能够自己画图并讲出这两次“换成顶点”；不必背一整页抽象凸分析。

3.3 旋转卡壳：为什么远侧指针能接着往前走

先在纸上做一个动作

拿两把直尺，保持平行，夹住凸多边形。每把尺子接触边界，整个多边形在它的一侧，这样的直线叫支撑线。两条平行支撑线接触到的一对顶点，叫一对对踵点。这个名词在这里就是“能够被一对平行尺子从两边夹住的顶点”。

缓慢转动两把尺子。接触点有时停在顶点上，有时沿边转移到下一个顶点。凸形外缘的方向有顺序，所以接触点也沿边界前进，不必反复退回起点。

算法利用的正是这个顺序。不能把任意乱序点列直接当作凸多边形使用；需要先求凸包，并让顶点沿边界一致地排列。

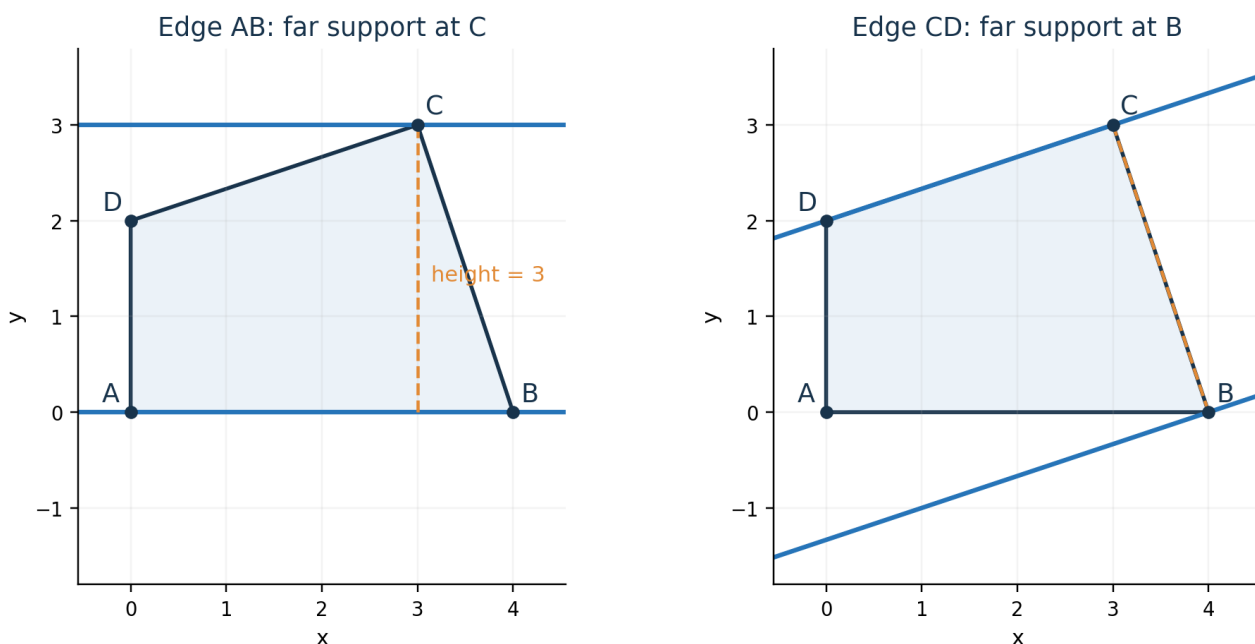


图 7: 两种方向下的平行支撑线与远侧顶点

图为教学补充坐标的原创示意。左图固定底边 AB ，远侧支撑点是 C ；右图固定边 CD ，远侧支撑点是 B 。两条支撑线之间的垂直距离帮助我们寻找候选点，最后仍要计算候选顶点之间的欧氏距离。

用叉积找到“离这一条边最远”的顶点

沿逆时针凸包取当前边 $v_i v_{i+1}$ ，令

$$e = v_{i+1} - v_i, \quad H_i(j) = \text{cross}(e, v_j - v_i).$$

$H_i(j)$ 是三角形面积的两倍。当前边长固定，因此它越大， v_j 到这条边所在直线的垂直距离越大。比较的是高度，暂时不是两个顶点之间的距离。

操作分成两步：

1. 沿凸包向前移动远侧指针 j ，直到再前进一步不能增加 $H_i(j)$ 。
2. 比较 $\|v_i - v_j\|$ 和 $\|v_{i+1} - v_j\|$ ，更新迄今最大的点对距离。

然后让当前边走到下一条边，远侧指针从刚才的位置继续走。标准的完整对踵枚举还要处理等高的相邻顶点；平行边会产生这种情况。不要将一般位置下的图例当成可以忽略平行边的理由。

手算一遍：为经典算法补充的四边形

本组整数坐标是教学补充数据，用于落实刚才的经典算法；不是 Toussaint 原图的坐标。逆时针顶点为

$$A = (0, 0), \quad B = (4, 0), \quad C = (3, 3), \quad D = (0, 2).$$

先只算底边 AB 。边向量是 $(4, 0)$ ，所以对点 (x, y) ，叉积为 $4y$ 。 C 给出 12， D 给出 8，故远侧点是 C 。这时检查

$$|AC|^2 = 3^2 + 3^2 = 18, \quad |BC|^2 = (-1)^2 + 3^2 = 10.$$

保留 18。继续转动，完整计算如下；使用距离平方是为了暂时不反复开方。

当前边	各顶点 A, B, C, D 的叉 积高度	远侧点	此步检查的距离 平方	目前最大值
AB	0, 0, 12, 8	C	$AC : 18, BC : 10$	18
BC	12, 0, 0, 10	A	$BA : 16, CA : 18$	18
CD	6, 10, 0, 0	B	$CB : 10, DB : 20$	20
DA	0, 8, 6, 0	B	$DB : 20, AB : 16$	20

最终 $D(P) = \sqrt{20}$ ，最远点对为 B, D 。你能独立复算表格的 CD 一行：边向量 $D - C = (-3, -1)$ ， $B - C = (1, -3)$ ，叉积为 $(-3)(-3) - (-1)(1) = 10$ 。

为什么能比两两枚举更快

两两枚举每次都从头检查所有搭配，计算量为 $O(n^2)$ 。卡壳让两侧接触点顺着凸包前进，扫描阶段为 $O(n)$ 。如果输入还没有凸包，先排序求凸包通常需要 $O(n \log n)$ 。

这个结论是对标准算法的相应阶段说的；实际程序若还要逐顶点核查全部约束，应把那部分成本另算。

学会到这里：能手算上表；能解释“叉积找远侧点，距离找直径”；知道指针单向前进来自凸性。七天内不要求你复述完整的摊还复杂度证明。

3.4 第二种任务：消防站建在哪里，最远的人家才尽量近

跟学 ETH 的经典选址例

ETH Zürich 的 Geometry: Combinatorics & Algorithms，附录 G 开头，用村庄消防站解释最小包围球：房屋位置已知，若路程按直线距离计、行驶速度相同，希望最远住户的到达时间尽量短，就把消防站放在最小包围圆的圆心。原讲义，印刷页 232，图 G.1

这里必须看清假设：不考虑道路绕行、不考虑交通拥堵、各方向速度相同。改变这些假设，就可能需要路网距离或带权选址模型。

先不求最优。你在地图上随便选一个站址 c ，测量它到每户的距离。最远那一户的距离为

$$R(c) = \max_i \|p_i - c\|.$$

以 c 为圆心、 $R(c)$ 为半径画圆，恰好能把所有房屋包住。现在移动 c ，让这个半径尽量小，便得到

$$R_* = \min_c \max_i \|p_i - c\|.$$

这就是最小包围圆问题。你可以直接把公式念成：“挑一个中心，让中心到最远的点的距离尽量小。”

两栋房屋：先用下界证明，再给出达到下界的方案

房屋在 A, B 。任何能包住它们的半径 r 、圆心 c 都满足

$$|AB| \leq |Ac| + |cB| \leq 2r.$$

所以 $r \geq |AB|/2$ 。以 AB 中点为圆心、半径 $|AB|/2$ ，恰好达到这个下界，因而最优。

请记住这种证明结构：任何方案都不能小于某数；我给出的方案达到这个数；所以最优。仅仅画一个看起来合适的圆，还不是最优性证明。

三栋房屋：两点决定的圆可能不够

原消防站例没有给具体坐标。下面是为它补充的教学手算实例：

$$A = (0, 0), \quad B = (4, 0), \quad C = (2, 3).$$

橙色虚线是以 AB 为直径的圆；蓝色圆是最小包围圆。图中 O 为蓝色圆的圆心。

第 1 步：先试刚学会的两点方法。最长边是 $AB = 4$ ，中点为 $(2, 0)$ ，直径圆半径为 2。可是 C 到该中点的距离是 3，因此它在圆外。不能因为找到最远点对，就说已经找到包住全部点的圆。

第 2 步：利用对称性。最优圆心在直线 $x = 2$ 上，设为 $(2, t)$ 。到 A, B 的距离相同，为 $\sqrt{4 + t^2}$ ；到 C 的距离为 $|3 - t|$ 。在本题最优位置两类最远距离达到平衡，于是解

$$4 + t^2 = (3 - t)^2, \quad t = \frac{5}{6}.$$

得到

$$c_* = (2, 5/6), \quad R_* = \frac{13}{6} \approx 2.1667.$$

第 3 步：为什么这次能断言最小。若仍取 $x = 2$ ，当 $t < 5/6$ 时，距 C 大于 $13/6$ ；当 $t > 5/6$ 时，距 A 或 B 大于 $13/6$ 。若圆心偏离对称轴， A, B 中至少一栋的水平距离会大于 2，而到 C 的

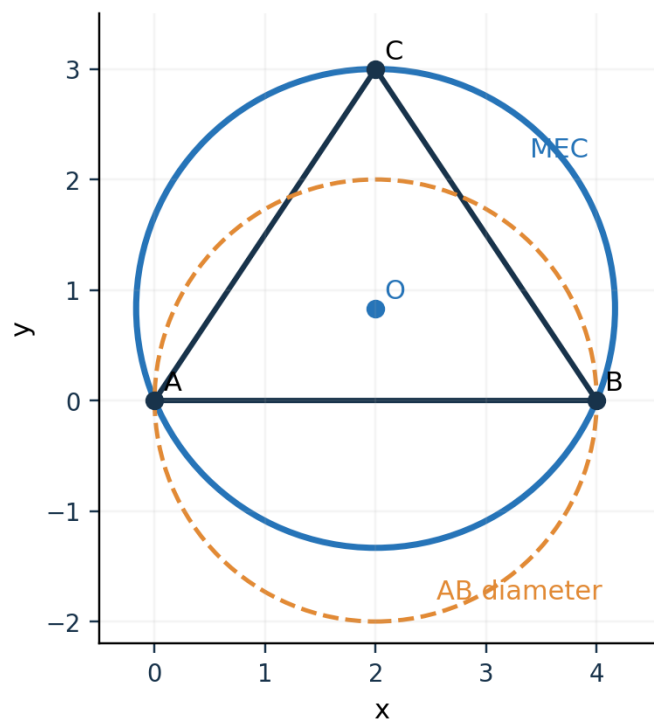


图 8: 同一组三点的直径圆与最小包围圆

距离也不会因水平偏移降低。因此不能得到更小半径。

该例三点的平均位置是 $(2, 1)$ ，最大距离为 $\sqrt{5} \approx 2.2361$ ，比 $13/6$ 大。求坐标平均值与控制最远距离是不同目标。

3.5 更多点时为什么只要关注少数“撑住圆”的点

想象一个圆被里面的点撑住，圆缩到再也缩不动。在二维中，总能找出至多三个点，单靠它们就能确定同一个最小包围圆。ETH 讲义称这样的最小决定子集为 basis；见印刷页 235，定理 G.4。原讲义先掌握二维的三种情况：

决定点	圆是什么	手算做法
一个点	半径为 0 的退化圆	圆心就是该点
两个点	以两点连线为直径的圆	取中点与半距离
三个不共线点	在需要三个支撑点时，为它们的外接圆	解两条垂直平分线的交点

不是“随便挑三个点作外接圆”。三点若构成钝角或直角三角形，最长边的直径圆已经足够；共线时也只需要两个端点。即使很多点恰在同一圆周上，仍可能只选出两个或三个点决定该圆。

一点点加入：随机增量思想

Emo Welzl 1991 年论文给出了著名的随机方法。其直观入口是：先解决已有点，再加入新点；新点在当前最小圆内，就不用改；新点在外，新的最小圆必须让这个点成为边界约束之一。作者公开论文

为何“新点在圆内就不用改”？原圆仍覆盖全部点；而加入点不可能让最优半径比原来更小，因此原圆继续最优。

新点在外时，把它作为一个已知边界点，再处理旧点。若又遇到破坏当前圆的点，就固定两个边界点；二维至多再由第三个点决定圆。随机顺序用来控制平均工作量，不表示允许漏点。

本课只要求你能跟踪两次分支：圆内则保留，圆外则重建。递归随机算法的期望复杂度证明放在深入阅读。你们程序使用固定种子的迭代实现，不能说它逐行照搬了 Welzl 论文里的递归伪代码。

3.6 本课的关键区别，合上书说一遍

问题	数学式	输出	不能由它直接推出什么
直径	$\max_{x,y \in P} \ x - y\ $	两个最远位置及距离	最远点中点能覆盖全部位置
最小包围圆	$\min_c \max_{x \in P} \ x - c\ $	一个中心及覆盖半径	这个中心是每种其他目标下的最佳中心

对任意非空有界点集， $D/2 \leq R_*$ 。证明就是两点距离不超过经圆心相连的两段半径。但这个不等式不必取等号，上一节三角形已经给了反例。

圆盘是凸集。它若包住凸多边形的每个顶点，就包住全部顶点的凸组合，也就包住整个多边形。所以区域的最小包围圆仍可只从顶点计算。

两道即时检查

1. 一条长 10 的线段，直径、最小包围圆半径分别是多少？
2. 一个区域面积为零，能否说明其最小包围圆半径很小？

解答。第 1 题为 10 和 5。第 2 题不能：很长的线段面积也为零。回答时要能画出这条线段，而不只是背“不能”。

3.7 课后应用：现在才把 Q1、Q2 放进来

下面把比赛题当成本课习题。前一章已经学过：每次有界角误差得到一个角楔，多次观测取交，得到源的可能位置区域 P 。

应用 3A：Q1 为什么要求直径，而清除又要看包围圆

题目。解释下列两个问题的差别：

1. 与观测相容的两个源位置，最远可能相隔多少？
2. 要把机器狗放在一个位置，保证距每个相容源都不超过 20 m，应该检查什么？

解答。问题 1 是直径。问题 2 要选同一个中心覆盖全部候选位置，是最小包围圆。若 $R_* \leq 20$ ，存在这样的清除点；反过来，若存在这样的点，也必有 $R_* \leq 20$ 。

必要条件 $D \leq 40$ 不足以替代它。边长为 40 的正三角形，直径为 40，最小包围圆半径为 $40/\sqrt{3} \approx 23.094$ ，不能用 20 m 半径覆盖。这是把标准反例的尺度改成题目的清除阈值。

应用 3B：在真实源码里找到刚才的每一步

以下行号对照仓库当前源码，路径从仓库根目录起算。

你刚学到的数学动作	源码位置	看这段时要指出什么
先求凸包	source/Q1/q1_solver.py, 144–157 行	排序去重；上下链；用叉积弹出不构成所需转向的点
以叉积找远侧点	同文件，160–176 行	i 是当前边； j 沿凸包前进；比较的是面积的两倍
比较真正的点对距离	同文件，177–181 行	依次比较当前边两端到 j 的距离平方，最后才开方
审核直径圆是否覆盖	同文件，184–198 行	用最远点中点、半直径作圆；再查所有顶点是否在圆内
两点定圆与三点定圆	source/Q2/q2_solver.py, 186–199 行	中点；垂直平分线等价方程；共线时返回 None
增量包围圆	同文件，202–235 行	固定种子打乱；圆内跳过；圆外重新约束；末尾重新量所有点的距离
对同一区域同时计算三项指标	同文件，272–275 行	直径、包围圆半径、面积分别计算，没有直接令 $R = D/2$

打开 Q1 求解器 和 Q2 求解器 后，只追踪这几段。先不把命令行解析、CSV 导入等外围细节混进几何算法。

特别留意 Q1: `diameter_circle_covers` 是检查直径圆的结果，并不是这个函数计算了一个最小包围圆。名字中的 `include_circle` 也不能当成已经裁剪场地圆的证据：201–204 行保留接口参数后就将它删除，Q1 实际是半平面交。

Q2 的包围圆最后用 `max(dist(...)) + 1e-7` 重算半径（235 行）。这使返回的中心与半径在浮点计算中覆盖输入点；它是数值覆盖复核，不等于用严格舍入证明该中心在所有浮点误差下仍精确最优。

应用 3C：自己运行一次标准小例，再打开比赛输入

在仓库根目录启动 Python，把下面内容逐行输入。这里先把通用例题送给你们的算法，目的在于确认“教材一步”与“代码一步”是同一个动作。

```
import sys
sys.path.insert(0, "source/Q1")
sys.path.insert(0, "source/Q2")
import q1_solver
import q2_solver

quad = [(0, 0), (4, 0), (3, 3), (0, 2)]
print(q1_solver.polygon_diameter(quad))

houses = [(0, 0), (4, 0), (2, 3)]
print(q2_solver.minimum_enclosing_circle(houses))
```


应得到直径约 4.472135955, 点对为 $(4, 0), (0, 2)$, 顺序可以相反; 三点圆心约 $(2, 0.8333333333)$, 半径约 2.1666667667。末尾比 $13/6$ 大约 10^{-7} , 对应刚讲过的覆盖裕量。

再打开 固定种子 Q1 结果, 找 `vertices_m`、`diameter_m` 和 `diameter_circle_covers`。不必背住随机坐标; 要会解释三个字段各回答什么问题。

评委追问练习

为什么没用两两枚举? 小规模时两两枚举完全可用, 也很适合作核对; 凸包顶点有序时, 旋转卡壳复用远侧指针, 扫描阶段较快。选择理由来自输入结构和重复调用成本, 不能回答“因为暴力法不正确”。

为什么不取多边形重心? 重心与最小化最大距离不是同一个目标; 消防站三点例已给出数值反例。清除要求覆盖全部可能位置, 因此选包围圆的中心更贴近条件。

这个算法每次都能得到有限圆吗? 对非空有限点集可以。Q1 的角楔交可能为空或无界; 必须先看状态, 不能把无界区域当成几个有限顶点硬算。

第 4 课 从“平均不错”走到“最坏也尽量好”

本课开始前，先能口述 $\min_c \max_i \|p_i - c\|$ 。如果你还分不清“先固定中心看最远房屋”和“再移动中心”，先重做消防站例。后面的 Q2 只是多了一层未来观测，基本阅读方式相同。

4.1 跟学教科书中的切比雪夫逼近

Boyd 与 Vandenberghe 的 Convex Optimization, 第 1.2.2 节、式 (1.6), 给出一个经典例子：选择参数，使若干表达式的最大绝对残差尽量小。第 6.1.1 节称它为 Chebyshev approximation, 也叫 minimax approximation。作者公开教材，印刷页 6-7、293

不用一开始就读矩阵。先取原例最简单的一维特例：用同一个数 x 去代表两个已知数 $a < b$ 。两个误差是 $|x - a|$ 、 $|x - b|$ 。你要解决

$$\min_x \max\{|x - a|, |x - b|\}.$$

这不是新编造一个题名：它是教材参数式在两个观测、一个常数参数时的特例。下面的推导是为了初学者重新展开的。

第一步：先把 x 固定，求它的“成绩”

暂且只看 $a \leq x \leq b$ 。左端误差是 $x - a$ ，随着 x 增大而上升；右端误差是 $b - x$ ，随着 x 增大而下降。这个 x 的成绩取两者较大值。

画出一条上升直线和一条下降直线，沿它们的上边界读值。上边界最低的位置就是两线相交处。因此

$$x - a = b - x, \quad x_* = (a + b)/2, \quad \text{最坏误差} = (b - a)/2.$$

若 $x < a$ 或 $x > b$ ，到更远端点的距离至少超过 $b - a$ ，显然不优。这也检查了刚才只在区间内分析的合法性。

第二步：把“最大值”改写成共同上限

引入一个误差上限 t ，要求每个误差都不超过它：

$$\begin{aligned} & \text{最小化 } t, \\ & \text{满足 } |x - a| \leq t, \quad |x - b| \leq t. \end{aligned}$$

还可拆成四个线性不等式。这种做法叫上图形式或辅助上界变量形式。先记住具体动作：把一个“最大值目标”变成“同一个上限限制所有项”，再把上限压低。

它是教材中的标准改写技巧。并不表示以后看到任意一个 $\min \max$ ，就一定能变成线性规划；还要看里面的函数和约束是什么。

4.2 极小极大的量词顺序

把消防站中心、常数拟合值都抽象成决策 x ，把可能的房屋或误差场景记作 u 。损失写作 $L(x, u)$ ，则

$$\min_{x \in X} \max_{u \in U} L(x, u)$$

要按以下顺序读：

1. 假定我已经选了某个 x 。
2. 在所有允许的 u 中，找这个 x 最糟糕的一项。
3. 更换 x ，让这个最糟糕值尽量小。

X 是允许选的方案集合， U 是需要防备的情况集合。若连续范围上最大值未必能取得，严格写法常用 $\sup$ ，即上确界。初学时先理解“盖住所有值的最小上界”。

顺序不能随便交换。先知道 u 再选择 x ，通常更容易。例如只有两个端点 a, b ：先选一个固定代表值，最坏误差是 $(b - a)/2$ ；若看到实际端点后才允许选择代表值，直接选该端点，误差为 0。这是信息获得时机不同，不是计算技巧不同。

“鲁棒”在这里需要什么假设

你知道不确定量属于一个集合，但没有把它当作已知概率分布。于是要求全部允许情况都满足条件，或者优化其中最坏的损失。

若知道可靠的概率模型，可以研究平均损失或某种风险水平。这些目标与最坏损失各有用途。现在只要会辨认：误差有上下界，不自动意味着它服从正态分布或均匀分布。

4.3 目标函数没有方便的导数，怎样找较好的点

课本中的常数逼近有解析解。但实际评价一个方案时，可能要先运行几何裁剪，再求包围圆；函数值可以算，导数却很难写。

这时进入无导数优化：依靠函数值比较寻找好方案。先学一维中最清楚的黄金分割法，再讨论它能用于什么程度的结论。

经典数值例：Cornell 开放教材的黄金分割 Example 1

Cornell 的 Computational Optimization Open Textbook, Derivative free optimization 页中 “Numerical Examples / Example 1: Golden Section Search” 给出的原例是

$$f(x) = (x - 2)^2 + 1, \quad x \in [0, 5].$$

原例链接

这里只采用原例函数与区间；以下数字重新计算。原网页在初始化和一次重命名处有排版问题，阅读时以公式 $[0, 5]$ 和下列推算为准。

这个函数显然在 $x = 2$ 时最小，最小值为 1。教材特意用一个已知答案的简单函数，方便检查数值算法是否走对。练习时暂时当作“只能代入函数，不能用求导直接解”。

先搞清删区间的依据

要求 f 在当前区间呈单谷形状：先下降后上升，唯一谷底，或符合相应单峰约定。在 $a < c < d < b$ 处取两个内部点。

- 若 $f(c) < f(d)$ ，最小点不可能在 d 的右侧，否则在走向谷底的下降段上应有 $f(c) > f(d)$ ；因此保留 $[a, d]$ 。
- 若 $f(c) > f(d)$ ，对称地保留 $[c, b]$ 。

等值情形在严格单谷假设下可保留含谷底的较小区间。代码用一种固定分支处理等值。

Michael Heath 的官方教学模块明确把单峰条件作为区间保留的前提，对应其教材第 6.4.1 节、算法 6.1、例 6.8。大学教学模块

为什么偏偏用 0.618

令

$$\tau = \frac{\sqrt{5} - 1}{2} \approx 0.618034, \quad c = b - \tau(b - a), \quad d = a + \tau(b - a).$$

它满足 $\tau^2 = 1 - \tau$ 。删去一边后，旧的一个内部点刚好落在新区间所需的黄金分割位置。所以后续每轮只需新算一个函数值。这个比例来自复用已计算位置的要求，不是经验选了一个神奇小数。

原例逐步计算

轮次开始	当前区间 $[a, b]$	c, d	$f(c), f(d)$	保留区间
1	$[0, 5]$	1.909830, 3.090170	0.008131, 2.188471	$[0, 3.090170]$
2	$[0, 3.090170]$	1.180340, 1.909830	0.671843, 1.008131	$[1.180340, 3.090170]$
3	$[1.180340, 3.090170]$	1.909830, 2.360680	0.008131, 1.130090	$[1.180340, 2.360680]$

把第二行的 1.909830 圈起来：它就是上一行的旧点，被再次使用。每轮保留的区间仍包含真正解 2。

自己动手。不要看答案，算新一轮的新点应在旧点 1.909830 的左侧还是右侧。答案：左侧；新区间内部点约为 1.631190 和 1.909830。因为右侧旧点仍被保留，只需补左侧新点。

学会到什么程度才够

你应能解释“为何可以删一段”“为何复用一个值”“为何需要单谷假设”，并算出前两轮。七天答辩准备不要求推导所有一维优化算法。

还要守住一条边界：若函数有多个谷，仅凭局部两个值删区间可能丢掉全局最佳点。先网格扫描、再细化若干好区间，是有用的数值策略；没有额外证明时，不能把它称作连续全域最优证明。

4.4 把三种层次分开，避免学名混乱

层次	要回答的问题	本课例子
建模	我决定什么，哪些情况必须考虑，什么叫好？	选 x ，控制最大绝对残差
数学结构	是否凸、是否单峰、能否换成等价约束？	常数切比雪夫逼近能转成线性约束
求解方法	实际怎样找数值答案？	解析求解、网格扫描、黄金分割

“极小极大”本身不是一段固定程序。“采用极小极大模型，再用有限网格与局部搜索近似求解”，才是一句完整说明。

4.5 课后应用：把 Q2 当成一份带约束的选址习题

现在只引入完成练习需要的题设：首次测向有 $\pm 1^\circ$ 误差；源的接收半径在 1000–1500 m 范围内；需要选择第二检测点，并评价第二次报告之后的不确定区域。20 m 是最终清除距离。

应用 4A：先说明决策与发生顺序，不急着列大公式

题目。Q2 中，哪个量由我们选？哪个量还不知道？什么时候才能选清除中心？

解答。我们先选第二站 s 。到那里测量后才得到报告 z 。根据这个报告形成新的相容区域 $P(s, z)$ ，然后才可以为该区域选一个覆盖中心 c 。

因此一次完整的评分流程是：

1. 暂定第二站 s 。
2. 暂定一种允许出现的报告 z ，形成 $P(s, z)$ 。
3. 为这个区域计算 $R(P) = \min_c \max_{g \in P} \|g - c\|$ 。
4. 改变报告 z ，找最坏的半径，作为站点 s 的成绩。
5. 比较不同的安全站点，选成绩最小的。

只在这五句话能说顺以后，再写压缩公式：

$$\min_{s \in \mathcal{S}} \sup_{z \in \mathcal{Z}(s)} \underbrace{\min_c \sup_{g \in P(s,z)} \|g - c\|}_{\text{该报告下的最小包围圆半径}}.$$

$\mathcal{S}$ 是安全第二站集合； $\mathcal{Z}(s)$ 是在站点 s 可相容的报告集合。这一式把消防站问题放在内层，再加“未来报告”“预先选站”两层。最外层的站点变量不等于最内层的清除圆心。

应用 4B：安全第二站为什么是“对所有可能源都能接收”

题目。第一次观测后，源可能在集合 F_1 中。采用统一的 1000 m 接收保证时，写出第二站条件。为什么不能只检查一个猜测的源位置？

解答。要使每个相容源位置都距离第二站不超过 1000 m：

$$\max_{g \in F_1} \|s - g\| \leq 1000, \quad \mathcal{S} = \bigcap_{g \in F_1} B(g, 1000).$$

每个可能源 g 都给出一个允许站点的圆盘。必须同时满足，所以取交。只满足其中某个圆盘，仅说明某个猜测的源可以接收。

这与消防站的区别很具体：消防站同时服务多个真实房屋；这里真源只有一个，但尚不知道哪个候选是真的，所以在保证性设计中需要照顾全部候选。

应用 4C：把安全条件手算成一段允许角度

题目。平移旋转坐标，把首站放在原点、首示向指向正 x 轴。用完整扇区 $0 \leq r \leq 1500$ 、 $|\eta| \leq 1^\circ$ 作为首测候选的保守范围。第二站离首站 L ，偏离首示向 ψ 。取 $L = 1000$ ，算统一接收保证下允许的最大 $|\psi|$ 。

步骤 1：先固定源的方向 η 。源坐标是 $g = r(\cos \eta, \sin \eta)$ ，第二站为 $s = L(\cos \psi, \sin \psi)$ 。余弦定理给出

$$\|s - g\|^2 = L^2 + r^2 - 2Lr \cos(\psi - \eta).$$

步骤 2：处理所有距离 r 。右侧是关于 r 的开口向上二次函数。因此在闭区间 $[0, 1500]$ 上，最大值在端点取得。只需检查 $r = 0$ 和 $r = 1500$ 。注意：开口向上函数的最大值在区间端点，最小值却可能在内部。

步骤 3：处理所有方向。在当前安全弧的角度范围内，最远扇区端点与第二站的夹角为 $|\psi| + 1^\circ$ 。于是同时要满足

$$L \leq 1000, \quad L^2 + 1500^2 - 2L(1500) \cos(|\psi| + 1^\circ) \leq 1000^2.$$

代入 $L = 1000$ ：

$$\cos(|\psi| + 1^\circ) \geq 0.75, \quad |\psi| \leq \arccos(0.75) - 1^\circ \approx 40.409622^\circ.$$

这里的“40.41°”是第二站相对首示向的允许偏角，不是两条观测射线在真实源处的交会角，更不是最佳站点答案。它只回答安全可行性，之后还要评价定位质量。

为何保留完整扇区？它简单且保守；实际场地圆可能截掉扇区的一部分，利用截断信息可进一步放宽某些候选。当前默认站点安全搜索没有声称已得到利用所有先验的最大安全域。

应用 4D：连续集合怎么变成程序可算的对象

题目。在某个固定第二站和固定报告下，要先求相容区域，再算圆。源码在哪些地方引入近似？

解答。程序用圆的外接多边形与角楔裁剪来表达区域；普通示向报告还意味着不在 5 m 近场内，但程序不挖去这个小孔，以保持凸性。因此计算集合是保守放宽。针对这个固定报告，包围放宽后的区域需要的半径不会小于包围真实区域所需的半径。

但接下来用有限个报告取最大值，是另一个动作。有限采样可能漏掉两个样点之间的峰值，所以不能因为“每个样点的空间集合是外包的”，就说“整个连续报告范围的最大值也被严格盖住”。

用两个符号记住方向即可：若 $P \subseteq \tilde{P}$ ，则 $R(P) \leq R(\tilde{P})$ ；而若 $Z_{\text{sample}} \subseteq Z$ ，则对同一个函数 f ， $\max_{z \in Z_{\text{sample}}} f(z) \leq \sup_{z \in Z} f(z)$ 。一个扩大空间集合，一个缩小检查的报告集合，不能混成同一条保证。

4.6 源码精读：照着刚才五步寻找函数

本节到代码才使用具体函数名；如果你还说不清应用 4A 的五步，先回去重做。

课堂动作	source/Q2/q2_solver.py 行号	应当读懂的输入、输出与分支
检查站点距整个扇区最远多远	<code>_sector_max_distance_sq</code> , 63–77	输入局部站点、扇区半径、角半宽；输出最大距离平方；代码也考虑反向方向落在区间内的情况
用统一 1000 m 条件筛选	<code>safe_uniform_1000</code> , 91–94	将上述最大值与 1000^2 比较
由固定 L 计算允许角度	<code>safe_angle_limit</code> , 97–111	应用 4C 中的反余弦公式
用外接多边形表示圆盘	114–121、147–154	顶点半径除以 $\cos(\pi/n)$ ；切线半平面逐个裁剪
得到首测候选区	<code>first_feasible_polygon</code> , 169–178	场地圆、1500 m 接收圆、首测角楔取交；不挖近场小孔
第二站和报告形成后验区域	278–284、318–330	先加第二接收圆，再加第二角楔；空集报告跳过
对报告找最大值	<code>robust_quality</code> , 333–400	先扫圆周报告角；再针对 R/D/A 各细化最多 8 个粗网格峰值；最终分别取最大
对站点找最小值	<code>optimize_fixed_baseline</code> , 408–469	固定基线，两侧偏角先粗后细；安全才评分；最后取最小

课堂动作	source/Q2/q2_solver.py 行号	应当读懂的输入、输出与分支
比较多条基线与两种目标	solve_case, 481-519	分别按 D 与 R 重新搜索，并对选出的站点加密复评

只读一小段，练习把代码译成人话

robust_quality 的核心评分在 326-330 行：

```
poly = post_region(base, second, key, delta_deg)
if not poly:
    return None
result = polygon_metrics(poly), poly
```

逐行说：用这次候选报告裁剪已有相容区域；如果交集为空，这个报告与当前放宽模型不相容，不参与评分；否则计算区域的直径、包围圆半径和面积。

再看 393-395 行：

```
row_R = max(records, key=lambda row: row[1].mec_radius_m)
row_D = max(records, key=lambda row: row[1].diameter_m)
row_A = max(records, key=lambda row: row[1].area_m2)
```

这是对每一种指标分别找最坏报告。最坏半径、最坏直径、最坏面积未必出现在同一个报告上，不应把三个数自动拼成某一个实际多边形的三项指标。

为什么代码里的黄金分割在求“最大值”

课堂例子求谷底；Q2 内层想找最坏报告，求的是峰顶。因此它在 380 行使用 `fc >= fd` 来决定保留方向。数学上，求 f 的最大值等价于求 $-f$ 的最小值。

它在粗网格候选峰附近迭代 34 次。迭代次数多，说明局部区间细；未证明该区间单峰、也未证明所有峰都已捕获时，不能由此宣称连续最坏值认证。

三个容易从函数名误读的地方

1. `safe_full_information` (80-88 行) 在文件里保留了另一安全域的实现，但默认 `solve_case` 使用 `uniform1000`。能够解释备选方法，不等于默认路径已经运行了它。
2. `refine_step_deg` 出现在 `robust_quality` 的形参中，当前函数体没有使用它。实际细化由粗峰区间和固定 34 次迭代控制；不能只改这个参数就声称报告采样变密。
3. `final_quality` 的默认报告步长是 0.1° ，但正常 `solve_case` 明确传入 0.2° ，`quick` 模式传入 0.5° (503-508 行)。回答“实际用了多少”应追到调用处。

这些细节留到第三天源码练习时读，不需要在第一遍几何入门时一起背。

4.7 用现成结果做一份“读报告”习题

打开固定种子 Q2 输入与 `quick` 结果。这是仓库已有的本地演示记录，不是外部教材数据，也不是官方成绩。

题目。仅根据文件回答：选了哪条基线？哪种目标得到什么站点？记录中的最坏半径约 55.82 m 能说明什么？

参考解答。本次 quick 搜索的 D 与 R 两种目标都选中 $L = 1000$ m、 $\psi = -33.25^\circ$ ；复评半径约 55.8199286 m，字段 `meets_20m_on_evaluated_reports` 为 `false`。该记录表示本次离散搜索与放宽模型的数值评价没有达到 20 m 判据。

它不能用来断言“所有连续候选站点都不可能达到 20 m”，也不能证明两种目标在全部场景都选择相同站点。更不能把有限报告的最大值叫作已认证的连续最大值。

方法选择的答辩练习

问：为什么要最小化最坏半径，不优化平均面积？

答：清除条件是一个位置能覆盖全部相容源，因此半径与清除距离直接对应。题设只给误差范围，没有给可信的报告概率分布，所以当前模型评价允许报告中的最坏情况。面积可以辅助比较形状，但长而窄的区域即使面积小，仍可能需要很大的覆盖半径。

问：为什么不直接用线性规划？教材刚才不是能化成线性规划吗？

答：课堂的常数逼近只有绝对线性误差，能拆成有限线性不等式。Q2 的外层站点会改变后验集合，评分还包含包围圆与连续报告最坏值，不能直接沿用那个简单线性规划。程序选择固定基线、两侧角度搜索并加密复评。

问：为什么不直接用梯度下降？

答：区域裁剪的顶点结构会改变，内层最坏报告也会切换，目标没有现成的光滑导数表达。当前每次可以直接评价函数值，故采用无导数数值搜索。这里没有给出梯度法一定失败的证明，也没有给出所用搜索的全局最优证明。

问：是否把最坏源位置与最坏报告独立拼起来了？

答：报告先通过后验区域非空检查；对应的源必须与该报告的角楔约束相容。代码是在所用的外包近似模型内检查相容性，因此还需承认圆近似与近场孔省略带来的保守放宽。

问：增加网格密度有什么用，是否足以证明全局最优？

答：有助于检查数值结果对分辨率是否敏感，也可能发现漏掉的峰或更好候选；多次结果接近仍属于数值证据。连续全域结论需要额外的界、全局结构证明或覆盖全部候选的认证方法。

4.8 本章过关标准

先盖住书，在纸上完成以下任务，再回看源码。

1. 画一个三角形，说明直径圆为什么会漏点；独立算出第 3 课三点例的圆心和半径。
2. 画两条平行支撑线，解释卡壳指针为何沿凸包前进；复算一次叉积高度和一次点对距离。
3. 用“固定方案—找最坏—换方案”讲清极小极大；举出先知道情况再决策会改变答案的端点例。
4. 完成黄金分割原例前两轮，指出单峰假设和复用旧函数值的位置。
5. 从 Q2 源码依次指出“安全站域、后验区域、单报告半径、最坏报告、最优候选站”各在什么函数中。

能做到这些，就达到了本章的入门与答辩要求。扩展阅读才包括随机算法的严格期望复杂度、一般鲁棒优化对偶理论和连续全域认证；它们不应挤占你先把上述五项真正学会的时间。

本章出处与使用范围

本章文字与推导重新编写。外部来源负责交代标准方法、经典问题与原例位置；补充的小坐标已在出现处标记。请按下面指定的小节阅读，不要求七天内通读整本原书。

资料	精确位置	本章怎样使用
Godfried Toussaint, Solving Geometric Problems with the Rotating Calipers, 1983	§ 1, 图 1, PDF 第 1–2 页；原文；作者说明	凸多边形直径的原始图例、支撑线及算法来源；整数四边形为教学补充
ETH Zürich, Geometry: Combinatorics & Algorithms, 附录 G	印刷页 232 的消防站例；235 的定理 G.4；236 的 § G.2；官方讲义	原有应用例、少量支撑点、增量思想；三栋房屋坐标为教学补充
Emo Welzl, Smallest Enclosing Disks (Balls and Ellipsoids), 1991	LNCS 555, 359–370；作者公开论文	随机算法的原始出处；不将当前迭代源码说成原论文原样代码
Stephen Boyd, Lieven Vandenberghe, Convex Optimization, 2004	§ 1.2.2, 式 (1.6)–(1.7), 印刷页 6–7；§ 6.1.1, 293；作者版 PDF	切比雪夫逼近参数式及共同误差上限的改写；两数据特例在本文重新展开
Cornell, Computational Optimization Open Textbook	Derivative free optimization → Numerical Examples → Example 1；原例	使用原函数 $(x - 2)^2 + 1$ 、原区间 $[0, 5]$ ，自行复算迭代数值
Jeffrey Naisbitt, Michael Heath, Golden Section Search	对应 Heath 第 2 版教材 § 6.4.1、算法 6.1、例 6.8；官方教学模块	明确黄金分割的单峰前提与区间保留理由

源码映射基于本仓库 `source/Q1/q1_solver.py` 与 `source/Q2/q2_solver.py` 的当前行号。课程没有修改算法；精确几何结论、浮点实现检查与有限搜索结果在课文中分别说明。

第 5 课 覆盖、旅行商与启发式：先分清要解决什么

本课的位置：第 4 天前半段。先用约 100 分钟学完 5.1–5.7，再用约 60 分钟做课后 B 题练习。需要会集合的并集、两点距离和加减法。本课不要求证明计算复杂性定理。

学完的标准：给你一组“谁能负责什么”，你能判断覆盖；给你一张距离表，你能算路线；给你一条现有路线和一个新任务，你能算插入增加多少路程。最后才解释自己的代码用了哪一步。

5.1 先用两个问题认识这门课

一个项目需要五种编程语言。每名程序员会其中几种。你要找尽量少的人，凑齐五种语言。这叫集合覆盖问题（set cover）：挑哪些集合，使它们合起来包含全部需求。

如果人已经选好，现在要依次去他们的住处收材料，再回学校，怎样走最短？这才是旅行商问题（traveling salesman problem, TSP）：全部地点已知，挑一个访问顺序。

这两个问题的输入、答案长得不同：

问题	输入	一个答案是什么	要优化什么
集合覆盖	需求清单；每个候选者能满足哪些需求	候选者名单	人数或选取成本
闭合 TSP	必访地点；两两路费或距离	访问次序，最后回起点	全程成本
开放访问路径	必访地点、起点及终点要求	访问次序，未必返回	全程成本

暂停自问：“用三个人能完成所有工作”回答了去这三家时怎样走吗？没有；它甚至还没有使用地址。反过来，“有一条很短的路线”也不能说明选中的三个人具备全部技能。

本课依次学“覆盖是否完成”“路线是否短”“信息后来才出现时怎样决策”。先把每个问题单独做懂，才把它们组合起来。

5.2 经典教材例题一：组建程序员团队

原题在哪里

Luca Trevisan 的 CS261 课程讲义《Lecture 4: TSP and Set Cover》§ 2 用下列编程团队说明集合覆盖。这里保留原例的人员与技能数据，中文叙述和下面的推算重新编写。[5-1]

需求是 C、C++、Ruby、Python、Java。候选者如下：

人员	C	C++	Ruby	Python	Java
Andrea	✓	✓			
Ben		✓			✓
Lisa		✓	✓	✓	
Mark	✓				✓

第一步：把“挑人”写成集合

把全部需求叫作全集 U ：

$$U = \{C, C++, Ruby, Python, Java\}.$$

每个人对应一个子集。例如 Lisa 对应

$$S_L = \{C++, Ruby, Python\}.$$

挑 Lisa 和 Mark，满足的需求是两个子集的并：

$$S_L \cup S_M = U.$$

并集中的同一种语言只记一次。两个人都会 C++，并不等于多满足一项新需求。

第二步：找到答案，并说明为什么已经最少

Lisa 与 Mark 能凑齐五种语言，因此两个人足够。表里没有任何一个人会五种语言，因此一个人不够。把这两句话合起来，才证明最少人数就是二。

这是一种很常用的证明结构：

1. 给出一个方案，说明“至多需要二”。
2. 排除更小的方案，说明“至少需要二”。
3. 上下界相等，得到最优值。

“我找到一个能用的方案”和“我证明它是最优方案”是两种不同的工作。答辩中也要分别说清。

第三步：学一个容易执行的通用算法

如果人员很多，不便枚举全部组合，可以每次选能补齐最多尚缺技能的人。这叫贪心集合覆盖。

←
←

在本例中，先选 Lisa，一次满足三项；剩下 C、Java，再选 Mark。执行过程是：

步骤	尚缺需求	选择	选择后剩下
1	五种语言	Lisa	C、Java
2	C、Java	Mark	空集

这里贪心恰好得到最优解。其终止条件“没有尚缺需求”保证覆盖完成；这一个例子的成功还不能证明贪心对所有输入都选人最少。一般算法的近似理论放在扩展阅读中，七天准备只需把操作与保证的区别讲清。

立即检查：第一轮之后再看 Andrea，为什么只算他新覆盖一项？因为 C++ 已经有了，只有 C 是新增贡献。

5.3 经典教材例题二：四个地点怎样走

原题在哪里

David Lippman 的开放教材《Math in Society》“Graph Theory and Network Flows”，印刷页 63–66，先枚举四点巡回，再用同一图讲最近邻。原边权如下；这里改用表格呈现。[5-2]

起点/终点	A	B	C	D
A	0	4	2	1
B	4	0	13	9
C	2	13	0	8
D	1	9	8	0

先读懂数据。 $d(A, B) = 4$ 是 A 到 B 的成本。表是对称的，因此反向成本相同。这里的权重是抽象路费，不能按图中线段的长短读取，也不能假定它来自平面直线距离。例如 $13 > 4 + 2$ ，所以它不满足三角不等式。

要求：从 A 出发，B、C、D 各访问一次，最后回 A，总成本尽量小。

方法一：穷举，得到可以核验的最优答案

先固定起点 A，剩下三个字母共有 $3! = 6$ 种排列。由于顺着走和倒着走成本相同，只需比较三条不同的巡回：

巡回	逐段相加	总成本
A-B-C-D-A	$4 + 13 + 8 + 1$	26
A-B-D-C-A	$4 + 9 + 8 + 2$	23
A-C-B-D-A	$2 + 13 + 9 + 1$	25

最优值是 23。这里“最优”的依据是所有情况都比较过；不是因为这条路线看起来比较自然。

若有 n 个地点，固定起点、合并逆序后有 $(n-1)!/2$ 条不同巡回 ($n > 2$)。地点增加时，枚举数量增长很快。例如 10 点已需要比较 $9!/2 = 181440$ 条。穷举适合教学小例和小规模核验，未必适合在反馈过程中反复执行。

方法二：最近邻，先学懂“贪心”究竟在贪什么

最近邻算法每一步选当前位置到尚未访问地点中成本最低的那个；全部访问后再回起点。

从 A 开始，亲手在表里划掉访问过的点：

当前点	还没访问的点及当前成本	本步选谁
A	B:4, C:2, D:1	D
D	B:9, C:8	C
C	B:13	B
B	全部访问完毕	回 A, 成本 4

因此得到 A-D-C-B-A，总成本 $1 + 8 + 13 + 4 = 26$ 。第一步省下来的钱，使后面必须承担 C-B 的高成本。局部最省的一步，没有自动包含全局最省的理由。

“贪心”是决策方式：每次按当前评价挑一个最好的；“启发式”说明我们用这种规则快速构造有用方案，但没有在这里保证全局最优。不要把“启发式”理解成没有算法、随便猜。

最优、近似保证、启发式：三个标签怎样用

标签	你必须拿出什么依据	本课的例子
精确求解	能证明输出就是最优值	四点穷举全部三种巡回
有近似比的算法	特定假设下，证明所得成本至多为最优的某个倍数	标准度量 TSP 的某些插入算法；本课只认识名称
启发式	清楚的可执行规则；通常还要实验比较效果	最近邻，或后面按绕行量选任务

一个算法可以既是启发式又在某些假设下有近似比。标签要跟随具体算法、成本模型与定理前提，不能跟随相似的名字。

5.4 第二个标准技巧：把新点插入现有路线

Cook 等的《Combinatorial Optimization》§ 7.2 专门介绍 TSP 插入方法。它们从小巡回出发，每次添加一点；不同版本在“选哪个新点”上有区别。[5-3]

先不背版本名称。只看一个动作：原来路线中的一段是 A-B，现在必须顺便访问 N，就把这一段换成 A-N-B。

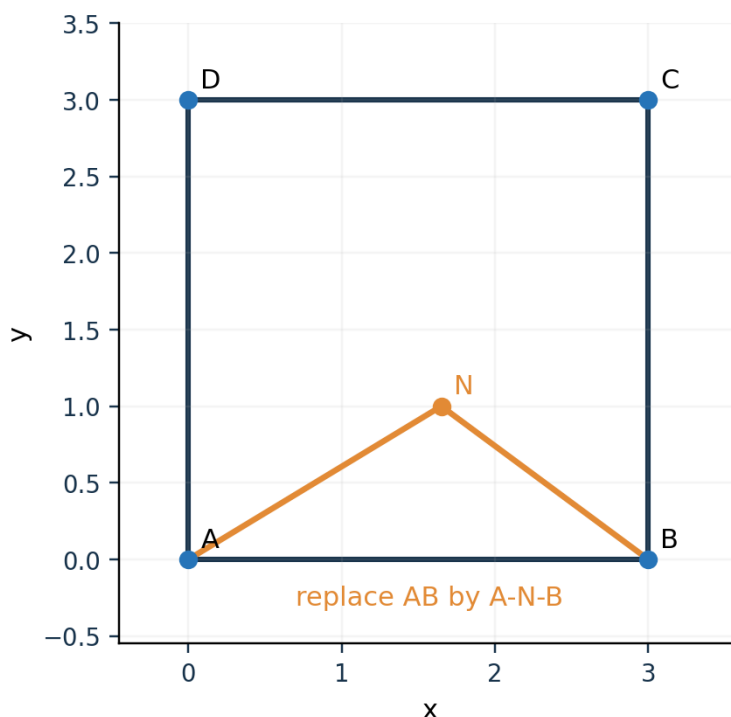


图 9: 插入一站替换现有路线的一条边

图是本课独立绘制的几何示意。蓝点为原路线地点，橙点 N 是新地点；它不声称复原某本教材的原图。

原来这一段花费 $d(A, B)$ ，替换后花费 $d(A, N) + d(N, B)$ 。路线其余部分没有改变，所以变化量是

$$\Delta = d(A, N) + d(N, B) - d(A, B).$$

减号的理由就是扣掉被拆掉的旧边。若忘了减旧边，就没有在比较“多花了多少”。

对于欧氏直线距离，三角不等式保证 $\Delta \geq 0$ ：绕经一个点不可能比直线更短；N 在 A-B 线段上时可等于零。对于不满足三角不等式的一般边权，变化量不必非负。

图中 $A=(0, 0)$ 、 $B=(3, 0)$ 、 $N=(1.65, 1)$ 。独立补算得到

$$\Delta = \sqrt{1.65^2 + 1} + \sqrt{1.35^2 + 1} - 3 \approx 0.6094.$$

用刚才的经典数据再练一次插入

以下是基于 5.3 原数据的课内延伸，并非声称原书给了这段插入解法。已有路线 A-C-D-A，现在把 B 插入。原路线成本为 $2 + 8 + 1 = 11$ 。

插入到哪一段	变化量	新总成本
A-C	$AB + BC - AC = 4 + 13 - 2 = 15$	26
C-D	$CB + BD - CD = 13 + 9 - 8 = 14$	25
D-A	$DB + BA - DA = 9 + 4 - 1 = 12$	23

选择 D-A 这一段，得到 A-C-D-B-A。对于这条已定小路线与这个新点，我们检查了全部三个插入位置，因此找到了最佳插入位置。这不等于在一般情形下证明全部点的最终路线全局最优。

几种经常被混叫的规则

规则	每一步究竟比较什么
最近邻	当前位置到每个未访点的成本
最近插入	先找离已有巡回最近的未访点，再选它的插入位置
最便宜插入	比较各个未访点、各个插入位置的增加成本
2-opt	已有完整路线，尝试换掉两条边以改进路线

七天内要会前面的变化量手算；2-opt 会说明“它是路线改进方法”即可。没有在自己的程序中找到对应操作，就不要在答辩中宣称程序实现了它。

5.5 为什么后来才出现的任务需要在线决策

“在线”在算法课里的意思是：输入逐步揭示，必须在知道全部信息之前行动。它与程序是否联网无关。

把四点 TSP 的距离表遮住一部分：你只知道 A、B，走到 B 才发现还有 C，而且到 C 后又可能发现 D。此时穷举当前已知地点，只能优化当前已知的子问题。未来任务会改变路线；以前已经走过的距离也无法撤销。

一个实际可行的思路是保存一条基础路线，有新任务时计算它是否顺路；若暂时太绕，就先继续基础路线。每次获得新信息后再计算。这是滚动决策的直观含义。

本课不展开所有在线算法理论。需掌握的是：

1. “未来所有地点都已知”和“新地点要靠观测才发现”是不同的输入条件。
2. 动态刷新分数，不等于已经求解全体未来情形的最优策略。
3. 启发式可以负责效率；任务完成与安全，还需另外的判据。

5.6 课内练习：先遮住答案

练习 5A。在编程团队原例中，若先选 Andrea，再选 Lisa，当前还缺什么？下一步必须补谁的一项能力？这个选择次序最终需要几个人？

练习 5B。四点原例从 C 出发执行最近邻。按“当前点—剩余点—选择”写出四步，并算总成本。它是否达到 23？

练习 5C。同一四点数据中，已有 A-B-D-A，要插入 C。逐一比较三个插入位置。

练习 5D。一家公司先选择仓库使每个客户至少被一个仓库服务，再给货车排仓库访问顺序。前半是哪种问题，后半是哪种问题？仅优化后半能保证仓库数量最少吗？

核答案与典型错因

- 5A：还缺 Java。Ben 或 Mark 都能补上，因此共三个人。说明选人次序会影响数量。
- 5B：C-A-D-B-C，成本 $2 + 1 + 9 + 13 = 25$ 。仍非最优。把所有起点都试一遍会改善某些例子，也不会因此自动成为精确算法。
- 5C：插入 A-B 增量 $2 + 13 - 4 = 11$ ；插入 B-D 增量 $13 + 8 - 9 = 12$ ；插入 D-A 增量 $8 + 2 - 1 = 9$ 。选 D-A，原长 14，新长 23。
- 5D：前半为覆盖选址，后半为访问排序。已固定的仓库数量不会因为换了访问顺序而减少。

做到这里，应能离开比赛题，单独讲清覆盖、TSP 和插入算法。下面开始把自己的题当课后应用。

5.7 课后应用 B-5：把 Q3、Q4 的工作拆开

应用题目

一台移动检测设备在平面寻找位置未知的多个源。它要到合适的地点检测，再走近已发现的源完成清除。问：集合覆盖与 TSP 思想各管哪一部分？源码是否把源的真坐标交给了旅行商算法？

解答第一步：先作对应

经典问题里的对象	本题里的对应	新增困难
全部必须满足的技能	圆域内任一可能源位置都必须有机会被发现	需求是连续区域里的无限多位置
一名成员覆盖的技能集合	一次停车检测能保证接收的源位置集合	Q4 还存在未知朝向
挑好后要访问的地点	覆盖站点，以及已发现的定位清除任务	后者在运行过程中才出现
固定城市坐标	已发现源的位置候选集合	没有准确坐标；定位服务会改变当前位置
路线总成本	移动、检测、换频道、清除的总时间	缩短距离未必使全部时间按相同比例减少

当前程序先构造有覆盖证明的站点集，再把定位任务穿插进访问过程。它没有运行经典的贪心集合覆盖来挑最少站点，也没有求解源真值坐标的完整 TSP。覆盖、服务排序、单源定位各有自己的任务和证据。

解答第二步：Q3 怎样获得覆盖保证

全向源最小接收半径为 1000 m。所以只要某站离源小于 1000 m，并检测对应频道，就一定会收到该源。把源可能的位置圆域记为 Ω ，站点集合记为 S ，希望满足

$$\Omega \subseteq \bigcup_{s \in S} B(s, 999).$$

这里 $B(s, 999)$ 是以检测站为圆心的保证发现圆盘，999 给 1000 留出余量。问题变成连续圆域覆盖。

默认构造是原点加八个环上站点。令目标圆半径 $R = 1800$ ，设计半径 $\rho = 995$ ，站点环半径为 a 。相邻外围站点夹角为 45° ，任一方向离最近站点方向不超过 22.5° 。外边界最难覆盖的位置满足余弦定理：

$$R^2 + a^2 - 2Ra \cos(\pi/8) \leq \rho^2.$$

解二次不等式，取较小根向内部移动 1 m，正是程序中的

$$a = 1800 \cos(\pi/8) - \sqrt{995^2 - 1800^2 \sin^2(\pi/8)} + 1 \approx 945.9734 \text{ m}.$$

中心 995 m 内由原点负责；对半径从 995 到 1800 的同一扇区，距最近外围站点的距离平方是径向变量的凸二次函数，在区间上最大值出现在端点。内端距离约 381.71 m，外端小于 995 m，因此整个扇区都被覆盖。八个扇区合起来覆盖外围环带。

本题要求你多处理的部分：经典团队例题只需逐项打勾；这里不能只在图上抽几个网格点打勾。代码另用圆弧区间检查边界及内部潜在孔洞，给定实际已扫描站点后再确认覆盖。完整实现见 Q3 covers_arena 第 187–205 行。

Q4 则不能仅靠近点圆盘，因为源可能背向设备。它采用 25 站；为什么还要让附近站点“包围”源，将在第 7 课证明。

解答第三步：读懂实际插入评分

设设备在 p ，下一覆盖站为 s 。某个已发现源的候选区被圆心 c 、半径 r 的圆包住。当前主线算

$$\text{score} = \underbrace{\|p - c\| + \|c - s\| - \|p - s\|}_{\text{假设访问中心的绕行量}} + r.$$

前三项就是 5.4 学过的插入变化量。再加 r ，是给“仍需定位、实际服务终点未必等于 c ”一个粗略惩罚。它与路程同为米，便于相加； r 不是已经证明精确等于后续定位所需路程。

代码在候选任务中选最小分数；不超过 600 m 才插入当前路段，否则先去下一覆盖站。已没有下一覆盖站时，按 $\|p - c\| + r$ 选任务。600 是策略参数，需要通过对照实验评价效果，不由经典 TSP 定理自动导出。

下面是本题公式的独立训练数据： $p = (0, 0)$ 、 $s = (1000, 0)$ 。

任务	候选圆中心	半径	距当前点	绕行量	最终分数
A	(500, 100)	30	509.902	19.804	49.804
B	(100, 500)	10	509.902	539.465	549.465

两个任务现在一样近，但 A 更顺着去下一站的方向，因此 A 先做。这个例子帮你理解为什么代码没有只按当前位置的距离排序。它是教学补算，不是正式比赛成绩。

解答第四步：把式子放回源码

行号按本仓库当前文件。Q3 本地与正式求解器前 502 行相同；Q4 前 208 行相同。

学过的动作	Q3 本地文件位置	Q4 本地文件位置
构造基础访问点	ring_sites 207–212; Config.ring_m=8 238; 原点扫描 471	coverage_route 85–89, 中心 +8 内圈+16 外圈
找已发现但未完成的任务	_choose 455	_choose_insert 181–182
计算三项绕行量	463	184
加不确定半径	465	183–184
选最小值并判 600 阈值	467–468; 参数 240	187
真正执行一次服务，再重新决策	run 479–487	run 194–200
宣告未知频道已排查完	_certify_and_prune 332–343	_finish_unknown_if_possible 130–139

源码：Q3; Q4。先找函数名，再找表中行号。正式 HTTP 文件是在相同策略后增加客户端，不是另一套路线算法。

代码精读：只看这六行，逐句翻译

Q3 第 462–468 行的关键语句：

```
t=self.tracks[i]; c,r=t.center,t.radius
detour=dist(self.p,c)+dist(c,next_site)-dist(self.p,next_site)
score=detour+1.0*r
values.append((score,i))
score,i=min(values)
return i if score<=self.cfg.insert_m else None
```

1. tracks[i] 是第 i 个频道的状态记录，取出当前位置外包圆。
2. detour 比较经中心和直接去下一站的距离。
3. score 加半径惩罚；系数 1.0 是这份策略的选择。
4. 把分数和频道号一起保存，之后才能知道要服务谁。
5. min 按元组顺序比较，先比分数；完全相等时频道号较小者优先。
6. 返回频道号表示“先服务它”；返回 None 表示本次没有值得插入的任务，主循环会去下一覆盖站。

这段代码只选择任务。它不计算新的方位角、不移动机器人、也不发清除请求；真正动作发生在 `_serve` 或 `_service`。

应用练习 B-5-1: 算法名能否这样写

请判断并改写下下列汇报句：

1. “我们用 TSP 保证搜索到所有干扰源。”
2. “我们采用最便宜插入，因此总时间不超过最优的两倍。”
3. “我们做了 2-opt，所以路线没有交叉。”
4. “代码里有 `rotate` 和 `prune`，正式主线会优化覆盖站。”

参考解答：

1. 发现完整性来自覆盖构造和每个未知频道的扫描；插入调度负责减少绕行。
2. 当前只比较当前路段，并加候选半径惩罚；任务在线出现、成本还包括测量等，因此不能套标准静态 TSP 的近似比。
3. 当前主线没有实现 2-opt。应说“采用在线插入评分”，不额外宣称不存在的优化。
4. Q3 Config 第 249–254 行中 `incidental=False`、`prune=False`、`rotate=False`、`speculative_r=0`。它们是保留的本地研究选项，默认入口 `Config()` 没启用。

应用练习 B-5-2: 为何路程短不一定全部时间最少

本地模拟器计时是

$$T = L/5 + 5M + S + 5C_{\text{succ}} + 3C_{\text{fail}},$$

其中 L 为米数， M 为检测次数， S 为换台次数， C 为清除次数。单位都是秒。请解释：一个改进减少 50 m 路程，却增加三次检测，不计其他变化，总时间会怎样？

解答：移动省 $50/5 = 10$ s，检测多 15 s，总共反而增加 5 s。源码依据为两套本地模拟器 `_move`、`measure`、`clear`；Q3 第 16–34 行，Q4 第 16–36 行。

5.8 答辩时的四个追问

问：为什么选择这个插入评分？

答：它直接估计把已发现任务插进当前搜索路段的额外移动量，再用候选半径粗略表示定位负担。每次只对已发现频道比较，计算便宜且能随反馈更新。选择它是为了效率；完成与清除安全分别由覆盖和几何证书负责。

问：为什么不直接求 TSP 最优？

答：经典 TSP 先给出全部固定地点。这里源位置未知，要边检测边发现，而且服务一个源会改变自己的位置与其他源的信息。即使精确解当前中心点的 TSP，也只是一个随观测变化的近似子问题。

问：为什么是 600，不是 500？

答：它是主线固定的启发式阈值。代码说明其作用，但没有证明 600 在所有实例上最优。比较其他值需要使用同样种子、同样题设与全部时间指标做对照实验。

问：所有频道都有候选多边形，是否表示每个频道都存在源？

答：没有。初始化候选区域表示“如果该频道有源，它可能在哪儿”。UNKNOWN 和 FOUND 是不同状态；收到正观测或 near 才证实存在。只有覆盖完成或已知存在数达到上限 16，才排除其余未知频道。

5.9 本课结束前的口头验收

请不用术语先讲，再补术语：

先选一些地点，使每个可能目标至少能被某次检测覆盖；这处理发现完整性。已经有一批要去的地点时，再排访问次序；这与旅行商有关。我们的任务是运行中才出现的，于是每次用绕行量加候选半径评分，决定是否插到下一覆盖站之前。这是在线启发式，没有全局最短保证。

能自己讲出来，并手算 5.3 和 5.4 的表，才进入第 6 课。

本课来源与原文入口

- [5-1] Luca Trevisan, CS261 Lecture 4: TSP and Set Cover, §2 “The Set Cover Problem”, 2011-01-21。使用 Andrea、Ben、Lisa、Mark 的原始团队例子；没有把课内新数据冒充其原题。<https://lucatrevisan.wordpress.com/2011/01/21/cs261-lecture-4-tsp-and-set-cover/>
- [5-2] David Lippman, Math in Society, 章节 “Graph Theory and Network Flows”, 印刷页 63–66（当前章节 PDF 第 15–18 页）。使用四点边权与穷举、最近邻原例；插入练习为本课在相同数据上的延伸。作者文件标注 CC BY-SA。<https://www.opentextbookstore.com/mathinsociety/current/GraphTheory.pdf>
- [5-3] W. J. Cook、W. H. Cunningham、W. R. Pulleyblank、A. Schrijver, Combinatorial Optimization, 第 7 章 §7.2 “Heuristics for the TSP”, 尤其 “Insertion Methods”。用于核对标准算法名称及定义；未移植原书历史实验纪录或近似保证到本题。MIT 课程节选：<https://math.mit.edu/~goemans/18453S17/TSP-CookCPS.pdf>

以上原文已于 2026-09-29 在线核对。原例之外的推导、图示和 B 题迁移为本教材独立讲解。一般算法定义与公开数据用于教学；不整段翻译复制原教材。

第 6-7 课 有误差时保留哪些可能，没看到时又能排除什么

本章的位置：第 4 天后半段学习第 6 课；第 5 天学习第 7 课及两组 B 题应用。需要会集合交、向量点积、凸组合、半平面和包围圆。若不懂这些词，先回第 1-3 课。

本章只围绕一个问题前进：一个观测没有直接给出真实状态，我们怎样把“仍可能的状态”保留下来？先用整数传感器手算，再走到平面定位。每次增加一个困难；比赛公式出现在课后应用中。

第 6 课 集合成员估计：答案可以是一个集合

6.1 为什么测到 4，不等于真实值就是 4

假设一个读数最多会错 1，读出 4。真实值可以是 3、4、5。你现在不知道哪一个是真的，却知道其他整数都不可能。这已经是一条有用结论。

把全部仍可能的值保存在一个集合里，叫作集合成员估计（set-membership estimation）；“成员”的意思是我们要保证真值仍属于这个集合。机器人教材也把这类对象称为非概率的信息状态。

它处理的是“误差被确定地限制在某个范围”的模型。另一条路线是给误差规定概率分布，计算不同状态的概率；那是概率估计。只有上界而没有分布时，不能凭空说 4 比 3、5 更可能。

6.2 经典教材例题：LaValle 的带扰动整数传感器

原例出处：Steven M. LaValle 《Planning Algorithms》§11.1.1, Example 11.7 “Sensor Disturbance”，印刷页 564。教材给出整数状态 x 、整数读数 y 、扰动 $\psi \in \{-1, 0, 1\}$ ，观测模型是 $y = x + \psi$ 。[6-1]

先把模型中的每个角色念清：

符号	角色	我们是否知道
x	真实状态	不知道，要估计
y	实际读数	知道，传感器返回
ψ	本次误差	只知道属于 $\{-1, 0, 1\}$

若收到 y ，逐个考虑允许的误差：

$$\begin{array}{c|ccc} \psi & -1 & 0 & 1 \\ \hline x = y - \psi & y + 1 & y & y - 1 \end{array}$$

因此与读数相容的状态集合为

$$H(y) = \{y - 1, y, y + 1\}.$$

这一步叫从观测反推相容状态。以后会看到“观测逆像”这个较抽象的词，它说的就是这件事。

6.3 同一教材模型，自己完成一次连续更新

下面的读数序列是本课基于原模型设计的纸笔练习，并非原书给出的数值序列。假定 x 不随时间变化，起初只知道 $x \in \{0, 1, 2, 3, 4, 5, 6, 7\}$ 。连续收到 4、5、6。

第一次读数 4，与它相容的是 $\{3, 4, 5\}$ 。第二次读数 5，与它相容的是 $\{4, 5, 6\}$ 。一个固定真值必须同时解释两次读数，因此取交集：

$$F_2 = \{3, 4, 5\} \cap \{4, 5, 6\} = \{4, 5\}.$$

再收到 6，相容集合为 $\{5, 6, 7\}$ ，所以

$$F_3 = \{4, 5\} \cap \{5, 6, 7\} = \{5\}.$$

时刻	新读数	仅看本次允许的状态	与旧集合取交后
0	无	无	$\{0, 1, 2, 3, 4, 5, 6, 7\}$
1	4	$\{3, 4, 5\}$	$\{3, 4, 5\}$
2	5	$\{4, 5, 6\}$	$\{4, 5\}$
3	6	$\{5, 6, 7\}$	$\{5\}$

为什么不能取并集？并集表示满足其中一次即可，会把 3 留下来；但 3 无法在误差不超过 1 时产生读数 5。多个同时必须成立的条件，使用交集。

为什么又不能直接平均读数？本例平均恰好是 5，这个巧合没有代替误差模型的证明。若真值 3，三次都读出 4，每次误差都是允许的 1，平均依然是 4。只有有界误差，不能承诺平均必然消除偏差。

6.4 把刚才的算法压成一条公式

旧集合叫 F_k ，新读数叫 y_{k+1} ，那么

$$F_{k+1} = F_k \cap H(y_{k+1}).$$

其中

$$H(y) = \{x : \text{存在一个允许误差, 使 } y = h(x, \psi)\}.$$

“存在一个允许误差”很关键：候选值只要能解释观测就保留；无需它在所有误差下都产生该读数。算法只需要三步：先设置初始集合；每次按模型构造相容集合；取交。正确性的核心不变量是

$$x_* \in F_k,$$

这里 x_* 是真实状态。如果初始集合含真值，而每次相容集合也含真值，交完后仍含真值。用数学归纳法重复这句话，就证明算法不会误删真值。

6.5 集合变空告诉我们什么

假设上例前两次读数为 4、5，第三次读出 8。此时

$$\{4, 5\} \cap \{7, 8, 9\} = \emptyset.$$

结论是没有任何一个固定整数能在规定误差界下解释全部读数。可能是误差模型不成立，可能状态已经变化，也可能数据或程序有错。不能一边丢掉旧信息，一边继续声称仍保持原来的安全证明。

在比赛程序中看到“empty feasible set”抛异常，就是这种不一致报警。它不是告诉你“目标被精确定位成不存在的一点”。

6.6 精确集合难存储时，用外包络

假设精确候选集合是两小块，存起来很麻烦。可以用一个更大的简单集合 P_k 包住它：

$$F_k \subseteq P_k.$$

如果以后始终保留这个包含关系，真值仍被保留。外包络可能含有一些已经不可能的点，代价是后续定位更慢或清除判据更保守。

一个熟悉的二维例子是凸包。若精确剩余集合为 A ，用 $\text{conv}(A)$ 替代，就把凹口、洞或分离块之间的一部分空间填回去了。它们全都在外包络中，但并不都是真实可行点。

这一段要达到的程度：能画“两小块”和“包住两块的凸多边形”，指着图说明“允许多留，不能误删”。不必在七天内学完区间分析与所有集合表示算法。

6.7 课内练习与解答

6A. 按同一整数传感器模型，连续读数为 4、4、4。最终集合是什么？重复读数一定使集合继续缩小吗？

6B. 把状态改为实数、误差改为 $[-1, 1]$ ，连续读数 4、4.6。手算集合更新。

6C. 已知 $F \subseteq P$ ，新观测的精确相容集为 H 。若程序输出 $P^+ \supseteq P \cap H$ ，能否保证 $F \cap H \subseteq P^+$ ？

解答。6A 为 $\{3, 4, 5\}$ ，重复相同约束不增加新信息。6B 为 $[3, 5] \cap [3.6, 5.6] = [3.6, 5]$ 。6C 能，因为 $F \cap H \subseteq P \cap H \subseteq P^+$ 。注意这里保证包含关系，不保证外包络面积严格减少。

课后应用 B-6 Q3 怎样把测向变成安全清除

B-6.1 题目：把整数换成平面位置

现在未知量变成源坐标 $g = (x, y)$ ，观测变成从检测点 p 指向它的方位读数 $\hat{\theta}$ ，角误差有界。请仿照 $H(y) = \{y - 1, y, y + 1\}$ 写出平面中的相容集合。

解答。如果真实方向在读数左右 δ 内，允许区域是从 p 出发的向前角楔。再利用成功接收给出的距离上界，有

$$H(p, \hat{\theta}) = \{g : |\text{wrap}(\arg(g - p) - \hat{\theta})| \leq \delta, \|g - p\| \leq 1500\}.$$

`wrap` 表示圆周上的最短角差，例如 359.9° 与 0.1° 相差 0.2° 。这与整数例题本质相同：把“能解释读数的所有状态”留下。

程序的计算半角为 1.0051° ，由物理误差 1° 、两位小数输出可能造成的 0.005° 舍入及 0.0001° 保护组成。物理误差仍是 $\pm 1^\circ$ ；额外小量属于计算保护。

B-6.2 为什么源码用了三个半平面

把读数方向单位向量记为 u ，侧向记为 v ，候选点写成 $g = p + xu + yv$ 。角楔可以写成

$$x \geq 0, \quad |y| \leq x \tan \delta.$$

两条角边界对应两个半平面；角宽小于 180° 时，它们的交已经选出向前区域。成功接收还允许利用距离上界 $R \leq 1500$ 。程序用

$$x \leq R$$

作为第三个半平面。得到的三角形包含真实扇形，但角点的欧氏距离略大于 R 。这样用统一的 `clip` 就能完成更新，保留凸多边形表示；这是外近似。

Q3 `bearing_update` 第 63–76 行先取 $R = \min(1500, \max_{z \in P} \|z - p\| + \text{EPS})$ ，再执行三次裁剪。它没有把一条精确方位射线当成观测，也没有精确保留圆弧边界。

B-6.3 无信号为什么也可能提供集合约束

Q3 的源为全向，且有效接收半径至少 1000 m。在一个确实存在且未清除的频道上，如果检测点 p 没收到信号，源不可能在保证接收的 1000 m 圆盘内部。

精确操作是从候选区挖掉开圆盘。挖完可能非凸，程序为继续用凸多边形，将其凸包作为外近似：

$$P^+ = \text{conv}(P \setminus B^\circ(p, 1000)).$$

Q3 `exclude_disk_hull` 第 78–98 行收集圆外顶点及边与圆的交点，再求凸包，并用略缩小的排除半径防止数值误删。如果圆盘完全在多边形内部，凸包可能把洞全填回；这次负观测就未必缩小当前表示。

对未知频道，一次无信号还可能表示本来无源。因此代码先把负观测保存下来；以后发现该频道的源时，再与这些历史约束一起更新。不能一次无信号就把整个频道标为 `ABSENT`。

B-6.4 为什么“小区域”可以成为清除证书

第 3 课学过最小包围圆。现在它的用途非常具体：如果候选区 P 被 $B(c, r)$ 包住，而真源属于 P ，那么真源距 c 最多 r 。

清除半径为 20 m，代码使用 19.8 m 作为认证阈值。如果 $r \leq 19.8$ ，直接去圆心是一个安全选择；但未必需要走到圆心。

任取满足 $\|q - c\| \leq 19.8 - r$ 的点 q ，对全部候选源都有

$$\|q - g\| \leq \|q - c\| + \|c - g\| \leq (19.8 - r) + r = 19.8 < 20.$$

因此认证可清除点至少包含圆盘 $B(c, 19.8 - r)$ 。代码选其中距当前位置最近的点，以少走一些路。注意这是全部有效清除位置的一个有保证的子集，未声称精确枚举所有可行清除位置。

纸笔检查。若 $r = 12$ 、当前位置距 c 为 100，安全小圆盘半径为 7.8，沿向 c 的直线走 $100 - 7.8 = 92.2$ m 就够。这是公式计算，不必估计真实源在圆内哪一点。

B-6.5 候选区还太大时，为什么选下一测点

Q3 在候选包围圆中心附近取两个侧移候选点：

$$q_{\pm} = c \pm 0.1r v.$$

侧向 v 垂直于当前位置到圆心的方向。程序先检查每个点到全部候选顶点的最大距离不超过 999 m，确保真实源一定处在接收范围；再用移动距离及下一覆盖站方向选其中一个。

从新地点得到角楔，与旧集合取交，就会缩小不确定性。这里“换地方”有意义；同一点反复测并不能保证带来独立误差样本。

进阶推导，第二遍读。成功读数的三角形外包络 $0 \leq x \leq R, |y| \leq x \tan \delta$ 能装入半径 $R/(2 \cos^2 \delta)$ 的圆。新测点最多偏离旧圆心 $0.1r$ ，所以它到旧候选集的最大距离至多 $1.1r$ 。下次包围圆半径可界为

$$r^+ \lesssim \frac{1.1}{2 \cos^2 \delta} r \approx 0.55017r,$$

其中数值实现另有微小容差。Q3 第 304–308 行还计算独立的三角形包围圆并复核全部顶点，防止只依赖增量圆构造的内部值。`_serve` 设 18 步上限，并在信号丢失或明显未收缩时抛异常；不把这种异常冒充定位成功。

B-6.6 逐项定位到代码

教材动作	Q3 函数与行号	读这一段时盯住什么
初始集合含全部可能位置	<code>arena_polygon</code> 58-61	外切 48 边形，不能换成内接多边形
相交更新	<code>bearing_update</code> 63-76	输入旧 <code>poly</code> ，输出裁剪后的 <code>poly</code>
负观测外近似	<code>exclude_disk_hull</code> 78-98	保留点、边圆交点、凸包
包围圆最终复核	<code>mec</code> 120-146	返回半径用所有原顶点重新求最大距离
选最近认证清除点	<code>nearest_certified_clear</code> 148-154	<code>slack=19.8-r</code>
把响应变成状态变化	<code>_observe</code> 281-311	<code>no_signal</code> 、 <code>near</code> 、 <code>direction</code> 三支
选择下一观测位置	<code>_tracking_point</code> 396-407	两侧候选、999 m 检查、位置评分
循环定位直到满足清除条件	<code>_serve</code> 409-430	半径门槛、实际动作、异常保护

文件：Q3 本地求解器。这部分在正式文件中具有相同行号。

B-6.7 应用练习与答辩追问

练习。一位同学说：“连续几个读数已经很接近，所以我对示向求平均，再取两线交点去清除。”请用本课模型指出他还缺哪个保证。

解答。近似读数或平均点不保证真值在 20 m 内；需要保留所有与有界误差相容的位置，或给出其他有效的最坏距离上界。当前代码用候选集外包络及包围圆，保证全部候选位置都能清除。

问：为什么不用 Kalman 滤波？

答：标准 Kalman 滤波通常围绕线性动态、噪声统计及均方误差工作；本题核心要求是确定误差界下保留真值并认证最坏距离。集合成员估计直接表达这种要求。可以研究概率方法辅助选点，但概率中心或协方差不能直接替代当前的确定性清除证书。

问：有没有实现后才发现的偶然成功？

答：保留真值、认证距离、覆盖发现是可以说明条件的几何论证；启发式节约了多少时间要用实验比较。不能拿若干局成功来代替全部前提下的证明。

第 7 课 方向盲区：同样的“没看到”，信息量可能不同

7.1 先离开比赛：照相机为何看不到身后

一个探测器只看前方半圆。在同样近的两个点中，一个在前面，另一个在背后，接收结果不同。于是“没看到”至少有两种原因：太远，或方向不对。

Ai 与 Abouzeid 的论文《Coverage by Directional Sensors》Fig. 1 给了一个明确的公开例子：同样四个定向传感器和六个目标，某配置启用四个仍漏两个目标，另一配置只启用三个却覆盖全部。它展示了朝向对覆盖的影响。[7-1] 原图没有提供可供本课手算的坐标表，所以这里不编一组坐标冒充它的原始数据。

下面用独立图示和点积手算，学会同一个标准扇区模型。

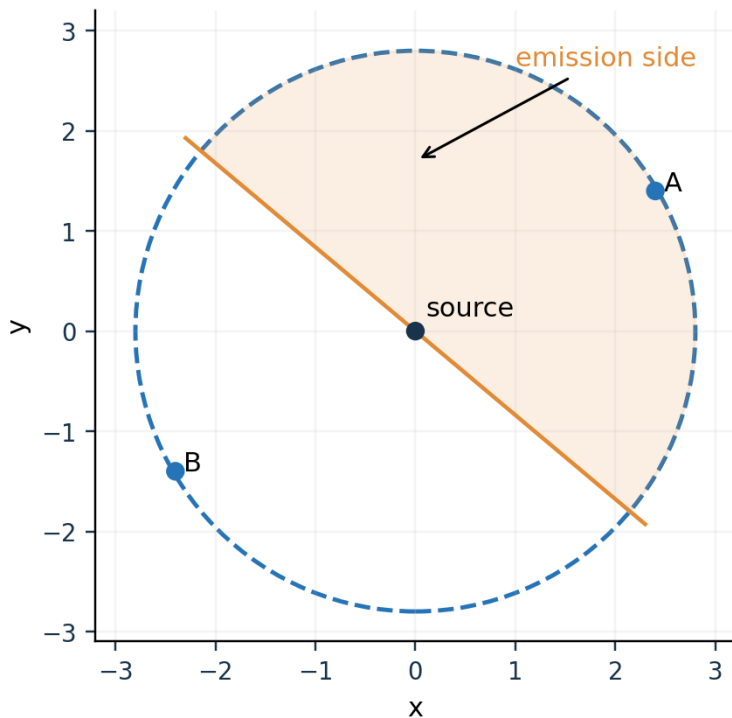


图 10: 同一距离范围内，A 在接收侧而 B 在背向侧

图中虚线圆是距离限制，浅橙半圆是接收侧，穿过源的橙线是真正的方向边界。A、B 都在距离范围内；A 有机会接收，B 在背向一侧。图为本教材独立绘制；不是原论文图的复制。

7.2 半圆怎样写成点积

源在 g ，单位朝向向量为 n ，接收点在 q 。向量 $q - g$ 从源指向接收点。

第 1 课的点积告诉我们

$$n \cdot (q - g) = \|q - g\| \cos \alpha,$$

其中 α 是接收点相对发射方向的夹角。前方左右各 90° 等价于 $\cos \alpha \geq 0$ 。所以半圆接收条件是

$$\|q - g\| \leq \rho \quad \text{且} \quad n \cdot (q - g) \geq 0.$$

第一个是不太远，第二个是在前半平面。二者同时成立才可接收。

独立手算例。 $g = (0, 0)$ 、 $n = (1, 0)$ 、 $\rho = 10$ 。点 $(3, 4)$ 距离 5、点积 3，接收；点 $(-3, 4)$ 距离也为 5、点积 -3，不接收；点 $(0, 5)$ 在边界，若模型含边界就接收。

要特别分清： n 是源朝哪边发射；测向返回的方向是接收点指向源，即 $g - q$ 。它们不是同一个向量。

7.3 正确解释负观测：先列出所有原因

对于一个已知存在、尚未清除的源，没接收到的条件是

$$\|q - g\| > \rho \quad \text{或} \quad n \cdot (q - g) < 0.$$

“或”关系不能只取前半句。若连源是否存在也未知，还需加“不存在”这个解释。

如果你设法让一次检测对全部候选位置都足够近，那么“距离太远”这个解释被排除了。这时负观测才必然来自方向盲区。主动选测点的一个目的，就是使反馈更容易解释。

仍然不能凭一个方向阴性立即得到源在前或在后；因为未知朝向还会造成多种解释。后面的成对方法会增加另一条证据。

7.4 一个通用几何事实：被站点包围，就不会全都在背面

假设目标 g 位于三个足够近的检测点 v_1, v_2, v_3 组成的三角形内。则有非负权重 $\lambda_1, \lambda_2, \lambda_3$ ，和为 1，使

$$g = \lambda_1 v_1 + \lambda_2 v_2 + \lambda_3 v_3.$$

这是“在三角形里”的凸组合表示。对任意朝向 n ,

$$\lambda_1 n \cdot (v_1 - g) + \lambda_2 n \cdot (v_2 - g) + \lambda_3 n \cdot (v_3 - g) = 0.$$

如果三个检测点全在严格背面，三个点积都小于零，它们的加权和不可能等于零。因此至少有一个点在接收闭半平面内。

先用图理解，再记公式：拿一条穿过三角形内部点的直线转一圈，不可能把三角形全部顶点都放到直线同一侧。这就是证明的几何含义。

距离条件还不能忘：如果三角形最长边不超过保证接收半径，内部任一点到每个顶点都不超过这个半径。因为到某顶点的距离不大于其他顶点距离的凸组合。于是“被包围”与“足够近”可以一起认证。

7.5 课内检查

7A. 三个检测点都在目标的左边，即使距离都很小，是否能保证任何未知朝向都收到？

7B. 目标在三角形内，但三角形某条边长 3000、有效距离只有 1000，刚才的发现保证还能直接使用吗？

7C. 读数 `no_signal` 的名字相同，能否自动沿用另一个物理模型的删除规则？

解答。7A 不能，目标可朝右。7B 不能，仅包围不足以证明距离。7C 不能，必须按当前观测模型重新写相容集合；标签的名字不决定信息量。

课后应用 B-7 Q4: 让方向盲区也提供可靠信息

B-7.1 第一道应用: 为什么 Q3 的负观测操作会失效

题目: 源在 $g = (100, 0)$, 朝正东, 接收半径 1000。机器狗在 $q = (0, 0)$ 检测无信号。如果套 Q3 排除以 q 为圆心的 1000 m 圆盘, 会怎样?

解答: 会把真源删掉。距离仅 100 m, 但 $n \cdot (q - g) = -100 < 0$, 它在背面。Q4_observe 第 121-129 行对普通 no_signal 不裁剪候选多边形; 这是有意保守。

B-7.2 第二道应用: 把三角形覆盖方法用于 25 站

Q4 的 coverage_route 第 85-89 行构造: 原点, 半径 995 m 的八点内环, 半径 1838 m 的十六点外环。

1. 外环正十六边形的内切半径为 $1838 \cos(\pi/16) \approx 1802.683 > 1800$, 因此把整个源圆域包住。
2. 用这些站点把大多边形剖成小三角形, 每条边不超过 995 m。
3. 任意源位于一个小三角形内, 三个顶点足够近, 且根据 7.4 至少一个顶点在源的接收侧。

这就把通用的凸组合事实变成了本题的完整发现保证。

第二遍阅读: 怎样核查“小三角形”不是一句空话。记内圈 I_k , 外圈 E_j , 中心 O , 下标按圈的点数循环。中央取 (O, I_k, I_{k+1}) ; 每个 45° 环带取下面三个三角形:

$$(I_k, E_{2k}, E_{2k+1}), (I_k, E_{2k+1}, I_{k+1}), (I_{k+1}, E_{2k+1}, E_{2k+2}).$$

合计 $8 + 8 \times 3 = 32$ 个。边长类型为 995、 $2 \cdot 995 \sin(\pi/8) \approx 761.540$ 、843、 $2 \cdot 1838 \sin(\pi/16) \approx 717.152$, 以及

$$\sqrt{995^2 + 1838^2 - 2 \cdot 995 \cdot 1838 \cos(\pi/8)} \approx 994.519.$$

全部小于 1000。此证明针对连续位置与朝向, 不用“随机抽样未发现漏点”替代。它也没有证明 25 站最少或访问次序最短。

原点、内环、外环的顺序影响时间; 完整覆盖看站点集合。外圈大于 1800 的原因是它要把源所在圆域包在内部; 题中源的活动区域与机器狗可走的区域不能混为一谈。

B-7.3 第三道应用: 保留一个曾经收到信号的位置

把最近一次取得正观测的位置记为锚点 a , 读数方向记为单位向量 u , 垂直方向记为 v , 候选源写成

$$g = a + xu + yv, \quad 0 \leq x \leq U, \quad |y| \leq x \tan \delta.$$

这里 $U \leq 1500$ 是以锚点为参照、保证真源距离不超过它的上界; 它不是源实际接收半径, 也不是包围圆半径。

锚点还有一个只写位置时容易忘的事实: 它确实收到过信号, 因此对未知朝向 n 有

$$n \cdot (a - g) \geq 0.$$

普通最近测点 last 可能无信号，不能替代正锚点 anchor。Q4 Track 第 97–100 行分别存这两个点和上界 u 。

B-7.4 第四道应用：设计一对测点

在角楔轴向的一半处，左右各偏移 $0.05U$ ：

$$q_+ = a + 0.5Uu + 0.05Uv, \quad q_- = a + 0.5Uu - 0.05Uv.$$

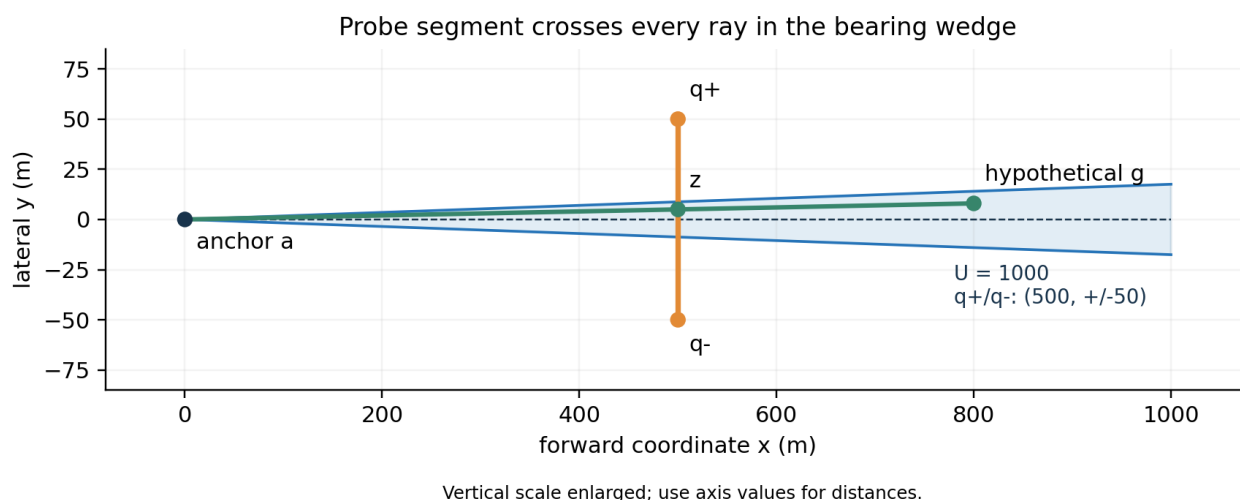


图 11: 成对测点横截角楔，假想远端源与正锚点的连线穿过探测线段

图是本课独立示意，取 $U = 1000$ 、锚点 $(0, 0)$ 、两探点 $(500, \pm 50)$ 、假想远端源 $(800, 8)$ 。为看清狭窄角楔，纵轴放大；距离按坐标刻度计算。图中的 z 是锚点到假想源的连线与探测线段的交点。

先检查两个条件，再谈收到什么结果：

条件一：都足够近。对角楔外包三角形内任一点，

$$\|q_{\pm} - g\| \leq U \sqrt{0.5^2 + (0.05 + \tan \delta)^2} \approx 0.504542U \leq 756.813 < 1000.$$

因此两点不可能因为太远而无信号。侧向差的上界为 $U(0.05 + \tan \delta)$ ，轴向差的上界为 $U/2$ ，再用勾股定理。图中侧向画在纵轴，轴向画在横轴。

条件二：两点之间的线段截住整个角楔。在 $x = 0.5U$ 处，角楔的半宽只有 $0.5U \tan \delta \approx 0.008772U$ ，小于两探点的半间隔 $0.05U$ 。所以每条可能的源方向射线都穿过两点之间。

这两个检查解释了 0.5 与 0.05 的作用。它们不是凭“左右各测一次应该更稳”就足够，必须验证距离和跨楔条件。

B-7.5 核心推理：为什么双负观测允许砍掉远半边

结论：在锚点确实正观测、两点足够近且跨住角楔、没有模型外漏检的条件下，如果两点都无信号，则真实源的轴向坐标 $x < 0.5U$ 。

先读直观版：如果源在探测线的前方，连接正锚点与源的那条线会穿过探测线段；正锚点已经位于接收侧。穿过的那一点沿着锚点到源的方向，也必须位于接收闭半平面里。但如果两个端点都在严格背面，中间整条线段都在背面，产生矛盾。

再把每一句写成可检查的式子：

1. 两点都足够近，所以双负意味着 $n \cdot (q_+ - g) < 0$ 、 $n \cdot (q_- - g) < 0$ 。对全向源，这种双负分支本来就不可能出现。
2. 反设 $x \geq 0.5U$ 。锚点到源的线与探测直线交于

$$z = a + 0.5Uu + \frac{0.5U}{x}yv.$$

3. 角楔条件给出 $|(0.5U/x)y| \leq 0.5U \tan \delta \leq 0.05U$ ，因此 z 在 $[q_-, q_+]$ 上。它是两个背向点的凸组合，所以 $n \cdot (z - g) < 0$ 。
4. 另一方面 $z - g = (1 - 0.5U/x)(a - g)$ 。括号非负，而正锚点满足 $n \cdot (a - g) \geq 0$ ，所以 $n \cdot (z - g) \geq 0$ 。矛盾。

由此可以安全保留闭半平面

$$P^+ = P \cap \{g : u \cdot (g - a) \leq 0.5U\}.$$

使用“ $\leq$ ”而非强行在浮点数里排除边界，是保守处理。Q4_dual_negative 第 155–158 行正是在做这一次半平面裁剪，然后更新上界及包围圆。

B-7.6 必须会说出的反例：为什么一个阴性不够

取 $a = (0, 0)$ 、 $g = (800, 0)$ 、 $U = 1500$ ，方向读数 0° ，两探点为 $(750, 75)$ 、 $(750, -75)$ 。令源的朝向

$$n = (-0.01, -\sqrt{1 - 0.01^2}).$$

锚点的点积为 8，因此有信号。上探点的点积约 -74.496，因此无信号；下探点约 75.496，因此有信号。源仍在 $x = 800 > 750$ 。如果收到上点一个阴性就裁到 $x \leq 750$ ，真实源会被删掉。

这是独立构造的数学反例，用来检验当前策略的必要前提；不把它包装成外部教材原题。

答辩时至少要能画出“正锚点、真源、一个背向测点和另一个正向测点”。只背“双负才能裁剪”而不知道单负反例，还没有真正掌握。

B-7.7 代码为什么常常只测一个点

_pair_round 第 159–169 行按下列顺序执行：

1. 用旧锚点、旧方向和旧 U 生成一对点。

2. 先去离当前设备较近的点。
3. 若返回 `direction`，更新位置集和正锚点，本轮结束。
4. 若返回 `near`，直接清除，本轮结束。
5. 只有第一点 `no_signal`，才去第二点。
6. 第二点仍 `no_signal`，才调用 `_dual_negative`。

“成对”表示设计与证据来自一对点，不表示无论结果如何都固定做两次检测。正观测已经能更新角楔，下一轮应该按新信息重建测点。

B-7.8 为什么这个过程会缩小，而不是原地死循环

双负分支裁到 $x \leq 0.5U$ ，结合角楔得到真源距原锚点

$$U^+ \leq \frac{0.5}{\cos \delta} U \approx 0.500077U.$$

正观测分支用新测点作锚点，旧候选区到它的距离已在 B-7.4 界为 $0.504542U$ 。因此更新后的 U 也至多约为这个量。两条未清除分支都把尺度降到约一半。

程序每次重算包围圆，达到 $r \leq 19.8$ 后，使用第 6 课相同的距离证书清除。代码设置九轮保护上限；这个数字是实现保护，不要把“程序设了上限”本身当作收缩证明。收缩理由是上面两个分支的几何上界，精确的浮点容差另由程序保守处理。

B-7.9 把方法、状态和代码逐项对上

要回答的问题	Q4 位置	具体证据
源在什么区域内初始化	<code>arena_polygon</code> 39–41	外切正 64 边形
正观测怎样变成角楔	<code>direction_update</code> 43–47	两条角边界+轴向上界
新锚点与 U 从哪来	<code>_positive</code> 109–112	只在正观测后更新
为什么单次阴性不裁剪	<code>_observe</code> 121–129	<code>no_signal</code> 返回但不改 <code>poly</code>
两点坐标怎么写	<code>_pair_round</code> 162–164	<code>LAMBDA=.5</code> 、 <code>ETA=.05</code>
什么条件才调用负裁剪	165–169	两次都 <code>no_signal</code> 且仍 <code>FOUND</code>
负裁剪究竟是哪条线	<code>_dual_negative</code> 155–158	<code>dot(u,g) <= dot(u,anchor) + .5*U</code>
什么时候允许清除	<code>_service</code> 170–178	先查半径，不够再继续成对探测
距离近能否保证接收	<code>_reuse</code> 140–147	近仅保证距离；方向可能仍无信号，故 <code>_observe</code> 不误裁

文件：Q4 本地求解器。前 208 行与正式文件一致。特别留意最后一行：`_reuse` 的 999 m 条件在 Q4 只解决距离，不保证避开盲区；它的安全性来自普通负观测不删除位置。

B-7.10 最后一组应用练习

练习 1。两个探测点都无信号，但它们距某候选源可能达到 1200 m。能否套用双负裁剪？

练习 2。两个点都足够近，但线段没截住全部角楔。能否套用同样结论？

练习 3。某次阴性之后，能否把该点写入 `anchor`，再进行下一轮？

练习 4。为何 `_reuse` 在一个停车点只选部分已发现频道复测，却不影响未知源的完整发现保证？

参考解答。

1. 不能，两个无信号可能来自超距；证明第一步失效。
2. 不能，假想远端源与锚点的连线可能绕过探测线段；交点不在两个负点之间。
3. 不能，锚点提供曾经接收的事实 $n \cdot (a - g) \geq 0$ ，阴性点不具备该事实。可以更新 `last`，不能冒充正锚点。
4. `_reuse` 用于机会补测已发现目标、提高定位效率；完整发现由覆盖路线上的未知频道扫描负责。两种循环承担不同职责。

第 6-7 课总验收：一口气讲清三件事

先讲通用方法。读数有误差时，保存全部能解释读数的状态；新读数出现就与旧集合相交。实现可用更大的简单集合包住精确集合，但始终必须保留真值。

再讲 Q3。方位读数变成角楔；全向阴性允许排除保证接收圆盘；包围圆给出最坏位置误差，半径足够小时才清除。

最后讲 Q4 的新增困难。阴性可能来自背向，因此普通阴性不直接裁剪。我们以正锚点为基准，设计足够近且横跨角楔的一对测点；双负说明源不在远半边，正观测则产生新的角楔。两条分支都收缩，再用同一个清除证书。

先会手算 6.3、7.2，再会画 B-7.5 的矛盾图，最后打开对应函数。达到这个次序，源码行号才会成为你已经理解的方法的位置提示。

本章来源与进一步阅读

- [6-1] Steven M. LaValle, *Planning Algorithms*, Cambridge University Press, 2006, Chapter 11, § 11.1.1, Example 11.7 “Sensor Disturbance”, p.564。整数状态、三值扰动模型来自原例；读数 4、5、6 和各训练序列为本课明确标注的独立扩展。作者章节：<https://lavalle.pl/planning/ch11.pdf>；作者全书入口：<https://lavalle.pl/planning/>。
- [7-1] Jing Ai, Alhussein A. Abouzeid, *Coverage by Directional Sensors*, WiOpt 2006, Fig. 1, § III-A, § III-B。四定向传感器与六目标的配置对照及标准扇区模型来源。作者高校稿：<https://sites.ecse.rpi.edu/~abouzeid/preprints/2006wiopt.pdf>。该稿封面页脚为 2005 草稿日期，会议出版年为 2006。

两个原始来源已于 2026-09-29 在线核对。本章的半平面推导、程序正确性分析与 B 题练习独立编写；没有声称某本通用教材直接给出了本题的 25 站或这份成对探测程序。那部分必须由题设和实际代码逐步推导。

附录 把课本方法讲到自己的程序上

到这里再打开源码。前四章每课先学经典问题、纸笔例题，再做 B 题应用；本附录只做源码导航与口头练习。遇到不会的函数，先回去做相应纸笔题，不用背函数名代替理解。

方法、题目与代码

下面按课文的顺序找函数。每条先读括号前的数学任务，再去找文件和行号，最后回答“哪个集合或状态变了”。

第 1-2 课：角度、半平面、凸包

Q1 文件为 `source/Q1/q1_solver.py`。第 257 行的 `_wedge` 把一次有误差的示向变成两个半平面；第 201 行的 `intersect_halfplanes` 求共同位置。第 144 行的 `_convex_hull` 整理边界，第 160 行的 `polygon_diameter` 找最远点对。要能说出空集、无界与有限多边形有什么不同。

第 3-4 课：圆与极小极大

Q2 文件为 `source/Q2/q2_solver.py`。第 202 行的 `minimum_enclosing_circle` 返回罩住全部候选顶点的圆；第 307 行的 `robust_quality` 搜索可实现报告中的最坏质量；第 408 行的 `optimize_fixed_baseline` 比较站点。要能指出内层是最坏报告，外层是候选站点。

第 5 课：连续覆盖与在线路线

Q3 文件为 `source/Q3/q3_local_solver.py`。第 207 行的 `ring_sites` 构造站点；第 187 行的 `covers_arena` 检查整个目标圆域的覆盖证书。第 454 行的 `_choose` 与第 470 行的 `run` 决定先扫描还是先服务。有限网格检查不能代替连续覆盖，启发式排序也没有最短路线证明。

第 6 课：集合估计与安全清除

仍在 Q3 文件中。第 63 行的 `bearing_update` 更新候选集；第 78 行的 `exclude_disk_hull` 用全向源的历史无信号作外近似；第 120 行的 `mec` 求覆盖圆，第 148 行的 `nearest_certified_clear` 选一个较近的安全行动点。每次更新必须留住真源。

第 7 课：定向盲区

Q4 文件为 `source/Q4/q4_local_solver.py`。第 159 行的 `_pair_round` 安排成对测点；第 155 行的 `_dual_negative` 只在两次都无信号时进入裁剪。定向单次无信号没有 Q3 的排除圆结论。

行号对应仓库原名的 source/ 文件。Q3/Q4 正式版本还有 HTTP 客户端；旧版 book/05_源码地图与逐段精读.md 可用于查询全部函数的起始行，不适合当入门课从头背。

四次从输入到输出的讲述

Q1。从 q1_solver.py:283 的 main 读取输入。solve_localization 把每次读数交给 _wedge 变成两条约束；intersect_halfplanes 求公共位置；polygon_diameter 求最远点对。自己画“输入一角楔—交集—凸包顶点—直径”五格图。评委问为何不直接让两条中心线相交，就画 $\pm 1^\circ$ 区间：误差让一个点变成一片可能区域。

Q2。q2_solver.py:481 的 solve_case 遍历 D/R 两个目标与候选基线；optimize_fixed_baseline 搜索站点方向；robust_quality 对该站点寻找可实现报告中的最坏区域，再以更细设置复查选中的站点。画出外层选站、内层报告的两层框。有限采样与局部细化给出数值结果，不能当成连续全域最优性定理。

Q3。q3_local_solver.py:470 的 Strategy.run 管整场；:281 的 _observe 处理一次测量反馈，可能更新频道候选集与包围圆；:313 的 _clear 更新清除状态。定位一个频道的安全证书与发现全部频道的覆盖证书是两件事。讲清“这个真源肯定在清除范围内”和“不会有尚未发现的源”分别由什么条件支持。

Q4。q4_local_solver.py:188 的 Strategy.run 安排扫描与服务。:170 的 _service 可能调用 :159 的 _pair_round；它从已有正观测的锚点安排两个位置。只有两次都无信号，才调用 :155 的 _dual_negative。单次无信号不能照搬 Q3 的排除圆：源可能近在旁边却朝另一个方向发射。

四轮闭卷演练

每轮先纸笔算，再对照课程答案、源码和 examples/ 的固定种子。最后不看书讲 90 秒。

1. 给四个点和一条有向边，算叉积，画凸包；再用直线裁掉正方形一角。说明 Q1 的 _wedge 提供哪两条边。
2. 给三个不共线点，找最远点对并画最小包围圆。给路线 $A \rightarrow B \rightarrow C$ 和新点 N，分别算 N 插入 $A-B$ 与 $B-C$ 增加多少距离。
3. 给两个站位方案，每个方案的两个可能报告各有损失：先各取最坏，再选较小者。指出 Q2 哪一层取最坏、哪一层优化。
4. 画全向源与定向源的无信号差异，用一个背向但靠近的源反驳“无信号一定远于 1000 米”。说出 Q4 何时才允许双负载剪。

每轮再自问：标准方法叫什么？它解决什么输入与输出？成立前提是什么？本题额外处理了什么？回答不出就回读本课经典例题。

被指到任意一行代码时

用五句话回答：这里读入什么；方法要解决哪一个标准问题；本题多了什么约束；这行如何改变集合、位置、状态或时间；结论属于证明、保守证书、数值近似还是一次实验。最后再说后面哪个函数使用这个结果。

能自己画最小例、算一步、找源码、说出前提，才算学会；只背得函数名不算。